MA2003B Application of Multivariate Methods in Data Science

Module 1: Interdependence Relationships

Instructor: Raul Gomez
Email: rgomez@tec.mx
Office: A4-123A

October 27, 2025

Table of contents

Part I

Univariate Normality Testing

Chapter 1

An Introduction to the Tidyverse

The Tidyverse is a collection of R packages designed for data science. It provides a consistent and user-friendly interface for data manipulation, visualization, and analysis. Although formally introduced as an ecosystem in 2016, many of these packages were developed by Hadley Wickham between 2007–2014.

1.1 Tidy Data

The core idea of behind the design of the Tidyverse is Wickham's definition of "tidy data" [3].

Definition 1.1.1. *Tidy Data* is a way of structuring datasets to make them easier to work with. In tidy data:

1. Each variable forms a column.
2. Each observation forms a row.
3. Each type of observational unit forms a table.

This structure allows for easier data manipulation, analysis, and visualization, as it aligns with the way data is typically processed in R. In this section we will describe the basics of the Tidyverse but, for more information on its use, the interested reader can check the following references: [5, 6, 7].

1.1.1 Tibbles and pipes

In the Tidyverse, data is often represented using a special type of data frame called a *tibble*. Tibbles are more user-friendly than traditional data frames, as they provide better printing and subsetting behavior. They also allow for more intuitive handling of data. In particular, tibbles are designed to work seamlessly with the pipe operator (`%>%`), which allows for chaining together multiple operations (usually known as "verbs" in the context of the tidyverse) in a clear and concise manner. To illustrate these ideas, in the next section we will perform data cleaning on the "WHO" dataset, to make sure that the associated tibble satisfies the tidy data principles. This is usually the first step in Exploratory Data Analysis (EDA) and is crucial for ensuring that the data is in a suitable format for analysis and visualization.

1.1.2 An Example: Tidying the WHO dataset

Before starting, we need to load the required libraries and load the WHO dataset:

```
library("tidyverse")
library("here")
library("cowplot")
library("patchwork")
library("krulRutils")
library("ISLR2")
library("magrittr")

options(scipen = 999) # Disable scientific notation
data(who)
```

We can now start our "tidying" process. The main problem with this dataset is that it contains variables (like "new_sp_m014") that are not very clear and that contain more than one piece of information. So we need to separate these variables and introduce better names for them and their associated values.

```
# We start by creating the variable "who_tidy"
# that contains the cleaned WHO dataset
who_tidy <- who %>%
  # We use pivot_longer to eliminate the variables starting with "new"
  # and use them as values instead
  pivot_longer(
    cols = starts_with("new"),
    names_to = "key",
    values_to = "cases",
    values_drop_na = TRUE
  ) %>%
  mutate(
    key = if_else(
      startsWith(key, "newrel"),
      sub("newrel", "new_rel", key),
      key
    ),
    cases = as.integer(cases)
  ) %>%
  separate(key, into = c("new", "type", "sexage"), sep = "_") %>%
  separate(sexage, into = c("sex", "age"), sep = 1) %>%
  select(-new, -iso2, -iso3) %T>%
  # Finally, we save the tidy data in both RDS and CSV formats
  saveRDS(here("data", "who_tidy.rds")) %>%
  write_csv(here("data", "who_tidy.csv")) %>%
  glimpse()
```

```
Rows: 76,046
```

```
Columns: 6
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
```

We now want to convert the columns "type", "sex", and "age" into factors. We start by constructing the following lookup tables:

```
who_type_lookup_tbl <- tibble(
  code = c("ep", "rel", "sn", "sp"),
  label = c(
    "Extrapulmonary TB",
    "Relapse case",
    "Smear-Negative pulmonary TB",
    "Smear-Positive pulmonary TB"
  )
)

who_sex_lookup_tbl <- tibble(
  code = c("f", "m"),
  label = c("Female", "Male")
)

who_age_lookup_tbl <- tibble(
  code = c(
    "014",
    "1524",
    "2534",
    "3544",
    "4554",
    "5564",
    "65"
  ),
  label = c(
    "0-14",
    "15-24",
    "25-34",
    "35-44",
    "45-54",
    "55-64",
    "65+"
  )
)
```

We can now append factor columns for the corresponding variables in the WHO dataset:

```
who_factor <- who_tidy %>%
  convert_codes_to_factor(
    code_col = type,
    lookup_tbl = who_type_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  convert_codes_to_factor(
    code_col = sex,
    lookup_tbl = who_sex_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  convert_codes_to_factor(
    code_col = age,
    lookup_tbl = who_age_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  glimpse()
```

```
Rows: 76,046
Columns: 9
$ country     <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan"~
$ year        <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997~
$ type        <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp"~
$ sex         <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f"~
$ age         <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014"~
$ cases       <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128~
$ type_factor <fct> Smear-Positive pulmonary TB, Smear-Positive pulmonary TB, ~
$ sex_factor  <fct> Male, Male, Male, Male, Male, Male, Male, Female, Female, ~
$ age_factor  <fct> 0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65+, 0-14, 15-24,~
```

Finally, we want to use this information to analyze the number of cases of tuberculosis by type and sex. We start by generating the corresponding frequency tibble.

```
who_type_sex_tbl <- who_factor %>%
  count(type_factor, sex_factor, wt = cases, name = "cases") %>%
  print(n = Inf)
```

```
# A tibble: 8 x 3
  type_factor                sex_factor    cases
  <fct>                      <fct>         <int>
1 Extrapulmonary TB          Female       941880
2 Extrapulmonary TB          Male        1044299
3 Relapse case               Female      1201596
```

```
4 Relapse case                 Male        2018976
5 Smear-Negative pulmonary TB Female      2439139
6 Smear-Negative pulmonary TB Male        3840388
7 Smear-Positive pulmonary TB Female     11324409
8 Smear-Positive pulmonary TB Male       20586831
```

1.2 The "Grammar of Graphics" and ggplot2

The concept of the "Grammar of Graphics" was introduced by Leland Wilkinson in his 1999 book *The Grammar of Graphics* [8]. This provides a framework for understanding how to create visualizations in a systematic way. It breaks down the process of creating a plot into components such as data, aesthetics, geometries, statistics, coordinates, and themes. The `ggplot2` package [4] implements this grammar in R, allowing users to build complex visualizations by layering these components. For more information on the Grammar of Graphics, its history and applications, the interested reader can consult the following references: [1, 2].

1.2.1 Layers in ggplot2

In `ggplot2`, plots are constructed by adding multiple *layers* that define the components of the visualization. Each layer corresponds to a specific aspect of the plot, and together they form the complete graphic. These layers include:

1. **Data:** The dataset to be visualized (usually a data frame or tibble). This is the source of the information for the plot.
2. **Aesthetics:** Mappings that relate variables in the data to visual properties of the plot, such as:
   - Position on the x- and y-axes.
   - Color, fill.
   - Size, shape.
   - Transparency.

   These mappings define *what* data is shown and *how* it is represented visually.
3. **Geometries:** Geometric objects that display the data, for example:
   - `geom_point()` for scatterplots.
   - `geom_line()` for line graphs.
   - `geom_col()` for bar charts.
   - `geom_histogram()` for histograms.

   The geometry controls *how* the data is drawn.
4. **Statistical transformations:** Optional calculations applied to the data before plotting, such as:

- `stat_bin()` for binning data in histograms.
- `stat_smooth()` for fitting smooth curves.

These are preprocessing steps for the data visualization.

5. **Scales:** Define how data values are translated into visual properties, for example mapping numeric values to colors or shapes. Scales also control axis ticks, legends, and guides.
6. **Coordinates:** The coordinate system used for the plot, such as Cartesian (`coord_cartesian()`), polar (`coord_polar()`), or flipped coordinates (`coord_flip()`).
7. **Facets:** Methods to split the data into subsets and display multiple plots arranged in a grid:
   - `facet_wrap()`
   - `facet_grid()`
8. **Labels:** `labs()` adds titles, axis labels, and captions.
9. **Themes:** `theme()` controls the overall appearance of the plot (fonts, background, gridlines).

Each layer can be combined and customized to build complex and elegant plots. This *layered grammar* allows you to think of your graphic as a composition of independent components rather than a single, monolithic object.

1.2.2 Plotting with ggplot2

We will illustrate these concepts by plotting the number of cases of tuberculosis by type and sex, using the frequency tibble we created earlier.

```
who_type_sex_plot <- who_type_sex_tbl %>%
  ggplot(aes(x = type_factor, y = cases, fill = sex_factor)) +
  geom_col(position = "dodge") +
  c_scale_fill("C rose", "C blue") +
  labs(
    title = "Tuberculosis Cases by Type and Sex",
    x = "Tuberculosis Type",
    y = "Cases",
    fill = "Sex"
  ) +
  theme_krul()
```

We can now save and plot the graph:

```
ggsave(
  filename = here("images", "who_plot.png"),
  plot = who_type_sex_plot,
  width = 12,
  height = 6,
  dpi = 300
```

```
)
who_type_sex_plot
```

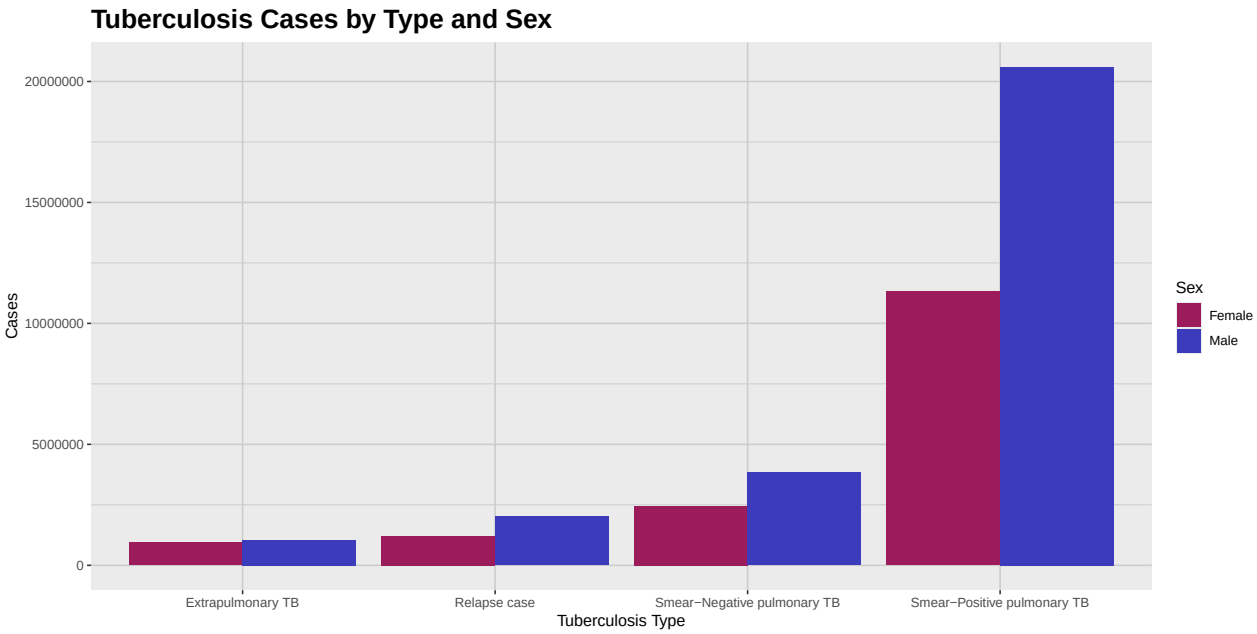

Figure 1.1: This plot shows a comparative analysis of the number of cases of tuberculosis by type and sex. As we can see from the plot, the number of cases is consistently higher for the male population across all types of tuberculosis.

1.3 Activity: Dataset Tidying

1. In the previous section we showed how to use the %>% operator to chain together multiple operations in the Tidyverse. In this exercise, you should show the result of all the intermediate steps in the tidying process of the WHO dataset, using the pipe operator.

2. The problem with the Billboard dataset, according to the principles of "tidy data", is that the weeks in which a song appeared in the Billboard chart are represented as columns, which makes it difficult to analyze the data. Using the techniques we have learned so far, the reader is required to tidy the Billboard dataset, so that it can be used to analyze the evolution of the ranking of the song "Who Let The Dogs Out" by "Baha Men" in the Billboard Hot 100 chart. The final visualization should look like the one shown below.

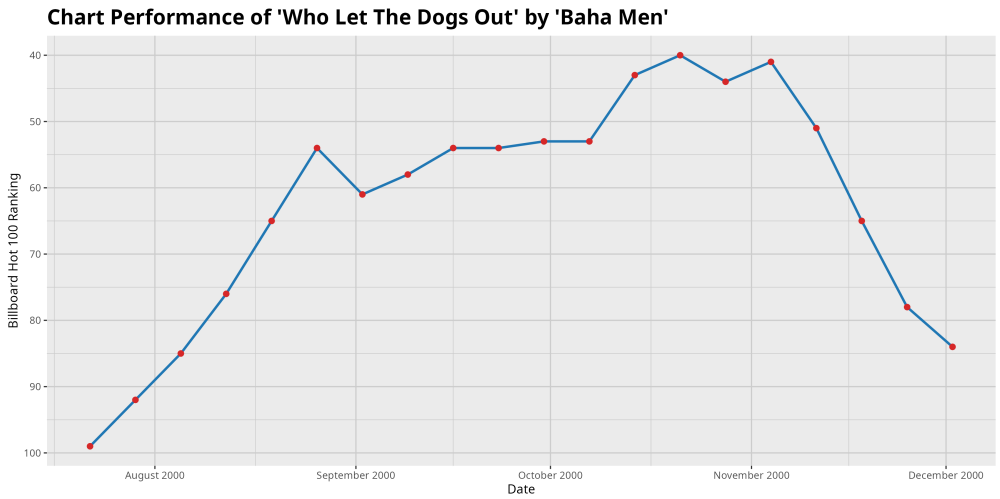

Figure 1.2: Evolution of the ranking of the song 'Who Let The Dogs Out' by 'Baha Men' in the Billboard Hot 100 chart. The song peaked at rank 40 in the week of October 21st, 2000, and remained in the top 100 for several weeks.

Bibliography

[1] Cleveland, W. S. (1993) *The Elements of Graphing Data*. Hobart Press.

[2] Hyndman, R. J., and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

[3] Wickham, H. (2014) Tidy data. *Journal of Statistical Software*, *59*(10), 1–23. https://doi.org/10.18637/jss.v059.i10

[4] Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag.

[5] Wickham, H., and Grolemund, G. (2017) *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media.

[6] Wickham, H. (2019) Functional programming with purrr. *UseR! Conference*.

[7] Wickham, H., et al. (2019) Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

[8] Wilkinson, L. (1999) *The Grammar of Graphics*. Springer-Verlag.

Chapter 2

The Standardized Moments

As it is well known, any normal distribution is completely determined by its mean and standard deviation. However, many real-world datasets do not follow a normal distribution. In this chapter, we will explore some other distributions and show how the concepts of skewness and kurtosis can help us understand its shape. We will also introduce QQ plots, which are useful graphical tools for comparing the distribution of a dataset against a theoretical distribution, which is usually taken to be normal. But before starting, let's load the necessary libraries and set some options.

```
library(tidyverse) # for data manipulation and visualization
library(broom) # for tidying data
library(here) # for file paths
library(patchwork) # for combining ggplots
library(krulRutils) # for custom package functions
library(magrittr) # for the pipe operator
library(e1071) # for skewness and kurtosis functions
library(car) # for robust scaling functions
library(ggforce) # for ggplot2 extensions
library(mvtnorm) # for multivariate normal distribution
library(MVN) # for multivariate normality tests
library(ellipse) # for generating covariance ellipses
library(ggrepel) # for better text annotations in ggplot2



options(scipen = 999)
```

2.1 Skewness

Definition 2.1.1. The *skewness* of a random variable X is defined as the third standardized moment, given by

$$\gamma_1 = \frac{\mu_3}{\sigma^3} = \frac{\mathbb{E}[(X-\mu)^3]}{\sigma^3},$$

where $\mu = \mathbb{E}[X]$ is the mean and $\sigma = \sqrt{\mathbb{E}[(X-\mu)^2]}$ is the standard deviation of X.

Skewness measures the asymmetry of a probability distribution. A distribution is said to be *positively skewed* (or right-skewed) if it has a longer tail on the right side, and *negatively skewed* (or left-skewed) if it has a longer tail on the left side. A perfectly symmetric distribution has a skewness of zero. To illustrate this idea, we will introduce the gamma distributions.

Definition 2.1.2. We say that a continuous random variable X follows a *Gamma distribution* with shape parameter $\alpha > 0$ and rate parameter $\beta > 0$, if its associated probability density function (PDF) is given by

$$f_X(x;\alpha,\beta) = \begin{cases} \dfrac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, & x > 0, \\ 0, & x \leq 0, \end{cases} \tag{2.1}$$

where $\Gamma(\alpha)$ denotes the Gamma function, defined as

$$\Gamma(\alpha) = \int_0^\infty t^{\alpha-1} e^{-t}\, dt.$$

Among other applications, the Gamma distribution is frequently used to model the number of total insurance claims over a given time period.

Remark 2.1.3. The skewness of a Gamma distribution is given by

$$\gamma_1 = \frac{2}{\sqrt{\alpha}}.$$

This means that the skewness is always positive, and it approaches zero as α increases. In other words, the distribution becomes more symmetric as α becomes larger.

We will now show how to plot the PDF of the Gamma distribution for $\alpha = 2$ and $\beta = 1$, highlighting the mean and median.

```
# Parameters
alpha <- 2
beta <- 1

# Create sequence of x values
x_data <- seq(0, 10, length.out = 200)

# Calculate density values
gamma_distribution_tbl <- tibble(
  x_data = x_data,
  density = dgamma(x_data, shape = alpha, rate = beta)
)

# Calculate mean and median
gamma_mean <- alpha / beta
gamma_median <- qgamma(0.5, shape = alpha, rate = beta)
```

```
gamma_sd <- sqrt(alpha) / beta

# Plot
gamma_distribution_plot <- gamma_distribution_tbl %>%
  ggplot(aes(x = x_data, y = density)) +
  geom_line(
    color = c_palette["C blue"],
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_mean,
    color = c_palette["C red"],
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_median,
    color = c_palette["C green"],
    linetype = "dotted",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = gamma_mean,
    y = 0.5,
    label = "Mean",
    color = c_palette["C red"],
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = gamma_median,
    y = 0.5,
    label = "Median",
    color = c_palette["C green"],
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = paste(
      "Gamma Distribution PDF (shape =", alpha, ", rate =", beta, ")"
    ),
    x = "X",
    y = "Density"
  ) +
```

```
  theme_krul()

gamma_distribution_plot
```

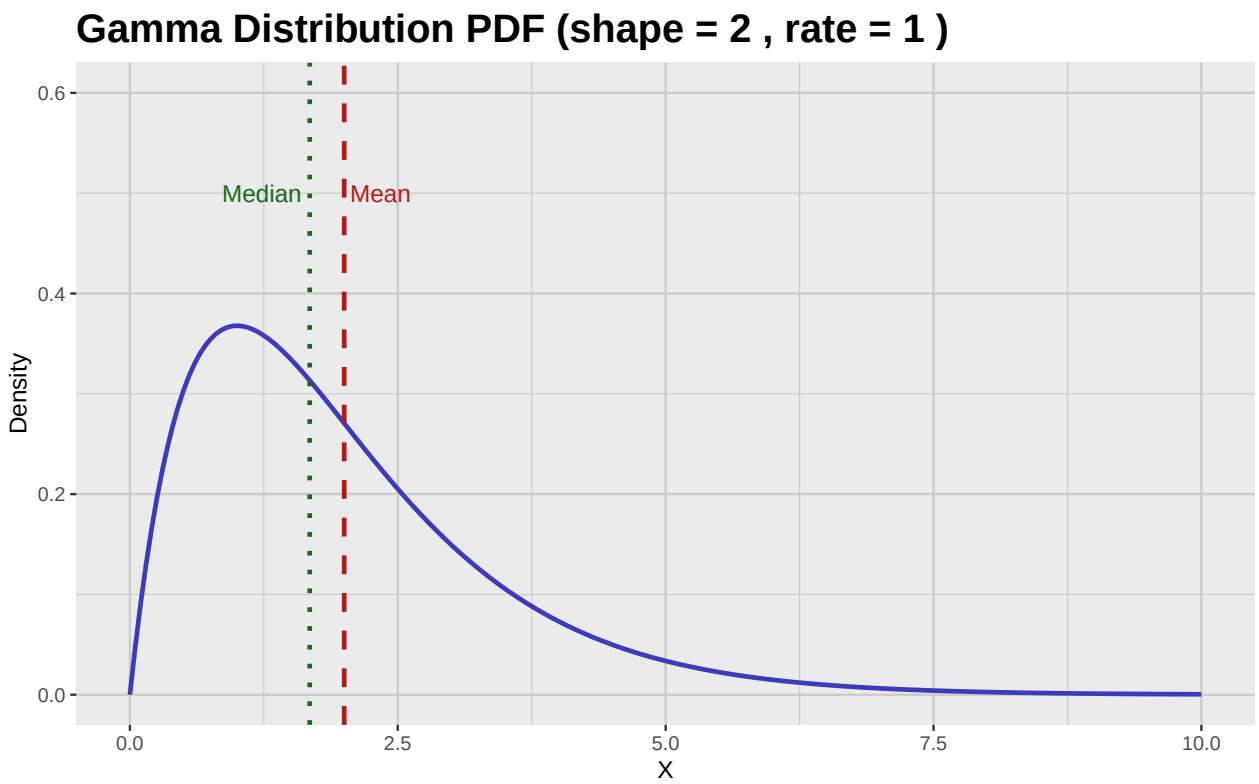

Figure 2.1: PDF of the gamma distribution with mean and median highlighted.

2.2 Kurtosis

Definition 2.2.1. The *kurtosis* of a random variable X is defined as the fourth standardized moment, given by

$$\beta_2 = \frac{\mu_4}{\sigma^4} = \frac{\mathbb{E}[(X-\mu)^4]}{\sigma^4},$$

where $\mu = \mathbb{E}[X]$ is the mean and $\sigma = \sqrt{\mathbb{E}[(X-\mu)^2]}$ is the standard deviation of X.

Kurtosis measures the "tailedness" of a probability distribution. A distribution with high kurtosis (also known as a *leptokurtic* distribution) has heavier tails and a sharper peak than a normal distribution, while a distribution with low kurtosis (also known as a *platykurtic* distribution) has lighter tails and a flatter peak. The kurtosis of a normal distribution is 3, which is why we are usually interested in computing the *excess kurtosis* which is given by:

$$\gamma_2 = \beta_2 - 3.$$

To illustrate this idea, we will introduce the Laplace distribution.

Definition 2.2.2. We say that a continuous random variable X follows a *Laplace distribution* with location parameter $\mu \in \mathbb{R}$ and scale parameter $b > 0$, if its associated probability density function (PDF) is given by

$$f_X(x; \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x - \mu|}{b}\right), \quad x \in \mathbb{R}. \tag{2.2}$$

Among other applications, the Laplace distribution is often used in image processing and computer vision, particularly in modeling noise in images.

Remark 2.2.3. The excess kurtosis of a Laplace distribution is given by

$$\gamma_2 = 3.$$

This means that the Laplace distribution has heavier tails than a normal distribution.

We will now show how to plot the PDF of the Laplace distribution with mean zero and variance one.

```
# Parameters
mu <- 0 # mean (location)
b <- 1 / sqrt(2) # scale

# Sequence of x values covering enough range to see tails
x_data <- seq(-5, 5, length.out = 300)

# Laplace PDF function
dlaplace <- function(x, mu, b) {
  1 / (2 * b) * exp(-abs(x - mu) / b)
}

# Create data frame with PDF values
laplace_distribution_tbl <- tibble(
  x_data = x_data,
  density = dlaplace(x_data, mu, b)
)

# Plot PDF
laplace_distribution_tbl %>%
  ggplot(aes(x = x_data, y = density)) +
  geom_line(color = c_palette["C blue"], linewidth = 1) +
  labs(
    title = "Laplace Distribution PDF (mean = 0, var = 1)",
    x = "x",
    y = "Density"
  ) +
  theme_krul()
```

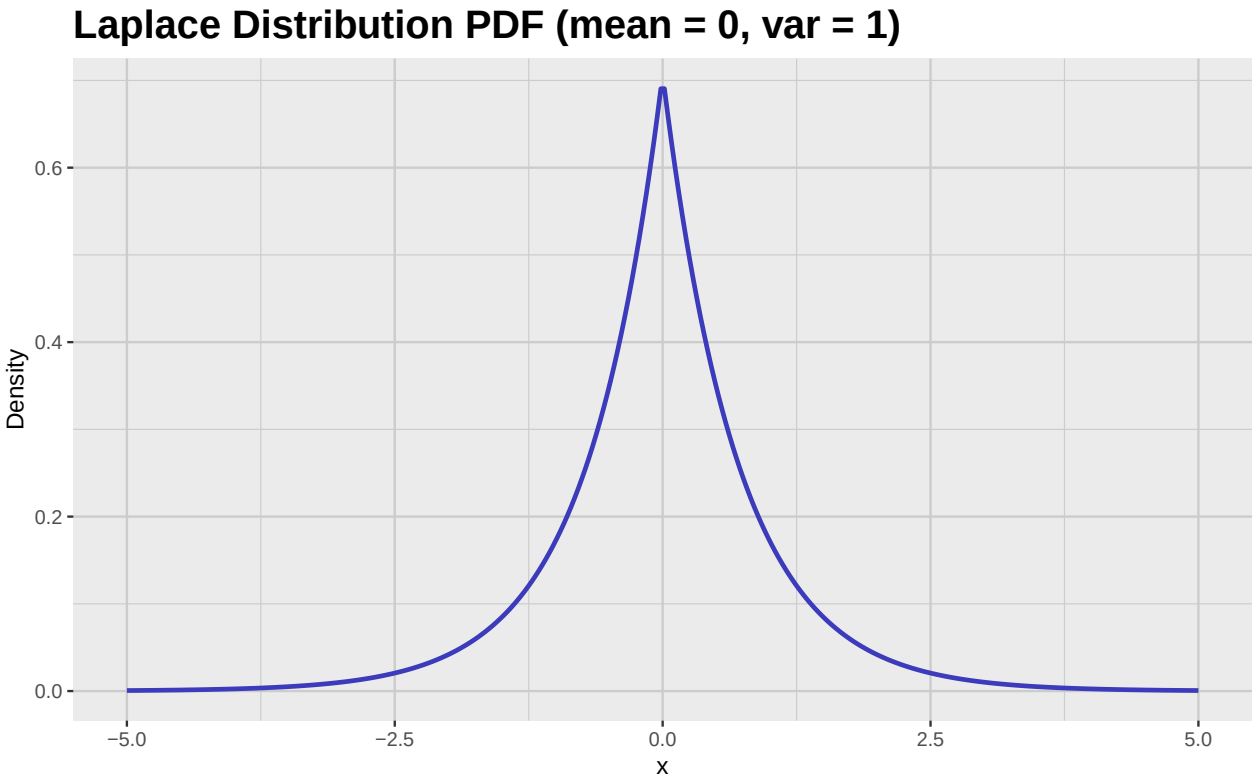

Figure 2.2: PDF of the Laplace distribution with mean zero and variance one.

2.3 QQ Plots

QQ plots, or quantile-quantile plots, are graphical tools used to compare the distribution of a dataset against a theoretical distribution, such as the normal distribution. They are particularly useful for assessing the normality of the data.

To construct a QQ plot, you should follow these steps:

1. **Sort the sample data:** Arrange the data in increasing order:

$$x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(n)}.$$

2. **Calculate plotting positions (empirical quantiles):** For each ordered value $x_{(i)}$, compute the corresponding probability

$$p_i = \frac{2i-1}{2n}, \quad i = 1, 2, \ldots, n.$$

3. **Find theoretical quantiles of the normal distribution:** Compute the theoretical quantiles from the inverse cumulative distribution function (CDF) of the normal distribution:

$$q_i = \Phi^{-1}(p_i),$$

where Φ^{-1} is the quantile function of the standard normal distribution.

4. **Plot the points:** Plot the pairs $(q_i, x_{(i)})$ with q_i on the x-axis and $x_{(i)}$ on the y-axis.
5. **Add a reference line:** Typically, we add a line through the first and third quartiles to help assess normality.

The closer the points lie to this line, the more the sample resembles the specified normal distribution.

To illustrate these ideas, we will now generate samples from the Gamma and Laplace distributions, and then we will construct their associated QQ plots and histograms. First we construct the samples:

```
set.seed(123) # for reproducibility

# Sample size
n <- 300
# Generate random samples from the Gamma distribution
gamma_sample <- rgamma(n, shape = alpha, rate = beta)

gamma_sample_tbl <- tibble(
  sample = gamma_sample
)

# Generate uniform random numbers in (0,1)
u <- runif(n)

# Apply inverse CDF of Laplace
laplace_sample <- ifelse(u < 0.5,
  mu + b * log(2 * u),
  mu - b * log(2 * (1 - u))
)

laplace_sample_tbl <- tibble(
  sample = laplace_sample
)
```

We will now construct the corresponding QQ plots and histograms. We will start with the Gamma distribution samples:

```
gamma_qq_plot <- gamma_sample_tbl %>%
  ggplot(aes(sample = sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
```

```
    title = "QQ Plot",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

gamma_histogram <- gamma_sample_tbl %>%
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogram",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()

gamma_plot <- gamma_qq_plot + gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Gamma Distribution: QQ Plot and Histogram",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

gamma_plot
```

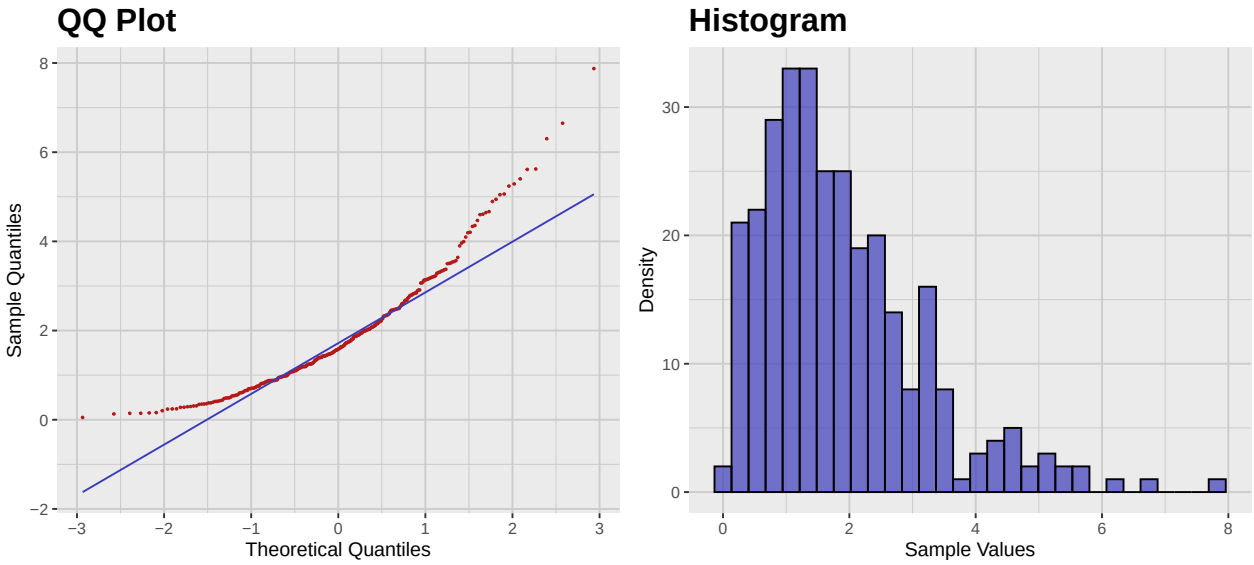

Figure 2.3: QQ plot and histogram of the Gamma distribution samples.

We will now construct the QQ plot and histogram for the Laplace distribution samples:

```
laplace_qq_plot <- laplace_sample_tbl %>%
  ggplot(aes(sample = sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plot",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

laplace_histogram <- laplace_sample_tbl %>%
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
```

```
  ) +
  labs(
    title = "Histogram",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()
laplace_plot <- laplace_qq_plot + laplace_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Laplace Distribution: QQ Plot and Histogram",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
laplace_plot
```

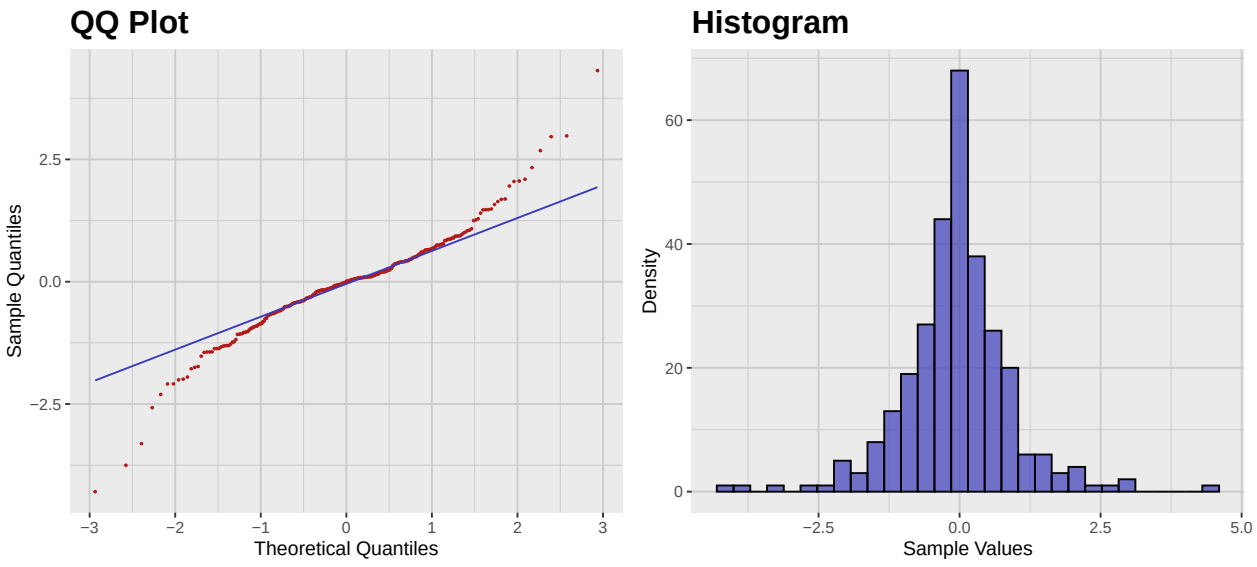

Figure 2.4: QQ plot and histogram of the Laplace distribution samples.

2.4 Sample Skewness and Kurtosis

So far we have defined skewness and kurtosis for random variables. However, in practice, we often work with samples rather than entire populations. Therefore, we need to define sample skewness and sample kurtosis.

Definition 2.4.1. Let $x_1, x_2, \ldots, x_n$ be a sample of size n. The *sample mean* $\bar{x}$ and *sample variance* s^2 are given by

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2.$$

The *sample skewness* g_1 is given by

$$g_1 = \frac{n}{(n-1)(n-2)}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{s}\right)^3.$$

The *sample excess kurtosis* g_2 is given by

$$g_2 = \frac{n(n+1)}{(n-1)(n-2)(n-3)}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{s}\right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}.$$

As an example, we will compute the sample skewness and sample excess kurtosis of the Gamma and Laplace samples we generated earlier. For the Gamma sample we have:

```
# Compute sample skewness and excess kurtosis for Gamma distribution
gamma_sample_tbl %>%
  summarise(
    sample_skewness = skewness(sample, type = 2),
    sample_excess_kurtosis = kurtosis(sample, type = 2)
  ) %>%
  glimpse()
```

```
Rows: 1
Columns: 2
$ sample_skewness        <dbl> 1.293772
$ sample_excess_kurtosis <dbl> 2.164353
```

For the Laplace sample we have:

```
# Compute sample skewness and excess kurtosis for Laplace distribution
laplace_sample_tbl %>%
  summarise(
    sample_skewness = skewness(sample, type = 2),
    sample_excess_kurtosis = kurtosis(sample, type = 2)
  ) %>%
  glimpse()
```

```
Rows: 1
Columns: 2
$ sample_skewness        <dbl> -0.07876743
$ sample_excess_kurtosis <dbl> 3.726374
```

2.5 Scaling and Standardization

In many situations, it could happen that the units we are using to measure different variables in our data are not compatible. For example, we could have a dataset that contains the height of

individuals in centimeters and their weight in kilograms. In such cases, it is often useful to *scale* the variables so that they are comparable. In this section, we will discuss the most common methods for scaling and standardizing random variables and samples.

2.5.1 Scaling Random Variables

Definition 2.5.1. An *affine transformation* of a random variable X is a transformation of the form

$$Y = aX + b$$

where a and b are constants.

Remark 2.5.2. In the context of statistics, affine transformations are often refered to as *scaling* (even when they also include a translational component).

Now assume that X is a random variable with mean μ and standard deviation σ. Then the random variable

$$Y = aX + b$$

will have mean $a\mu + b$ and standard deviation $|a|\sigma$. In particular, if we choose $a = 1/\sigma$ and $b = -\mu/\sigma$, then the random variable

$$Z = \frac{X - \mu}{\sigma}$$

will have mean 0 and standard deviation 1. This particular affine transformation is called a *standardization*, and the resulting random variable Z is called the *standardized* version of X. Z is also sometimes called the *z-score* of X.

Remark 2.5.3. Notice that if X is not normally distributed, then Z will *not* be the standard normal distribution. Standardization does not make a non-normal distribution normal.

Observation 2.5.4. Notice that, by definition,

$$\gamma_1(Z) = \mathbb{E}[Z^3] = \frac{\mathbb{E}[(X-\mu)^3]}{\sigma^3} = \gamma_1(X)$$

and

$$\gamma_2(Z) = \mathbb{E}[Z^4] - 3 = \frac{\mathbb{E}[(X-\mu)^4]}{\sigma^4} - 3 = \gamma_2(X).$$

As an example, we will consider again the Gamma distrubution that we introduced earlier. In this case we have that

$$\mu = \frac{\alpha}{\beta} \quad \text{and} \quad \sigma = \frac{\sqrt{\alpha}}{\beta}.$$

Using this information, we can generate the plot of standadized version of the Gamma distribution and compare it with the original one.

```
# Calculate density values
z_gamma_distribution_tbl <- gamma_distribution_tbl %>%
  mutate(
    z_data = (x_data - gamma_mean) / gamma_sd,
    z_density = dgamma(x_data, shape = alpha, rate = beta) * gamma_sd
```

```
  ) %>%
  select(z_data, z_density)

# Calculate mean and median
z_gamma_mean <- 0
z_gamma_median <- (gamma_median - gamma_mean) / gamma_sd

# Plot
z_gamma_distribution_plot <- z_gamma_distribution_tbl %>%
  ggplot(aes(x = z_data, y = z_density)) +
  geom_line(
    color = c_palette["C blue"],
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_mean,
    color = c_palette["C red"],
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_median,
    color = c_palette["C green"],
    linetype = "dotted",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = z_gamma_mean,
    y = 0.55,
    label = "Mean",
    color = c_palette["C red"],
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = z_gamma_median,
    y = 0.55,
    label = "Median",
    color = c_palette["C green"],
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = paste(
```

```
      "Standardized"
    ),
    x = "X",
    y = "Density"
  ) +
  theme_krul()

gamma_distribution_plot <- gamma_distribution_plot +
  labs(
    title = "Original"
  )

combined_gamma_plot <- gamma_distribution_plot + z_gamma_distribution_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Gamma Distribution: Original and Standardized",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

combined_gamma_plot
```

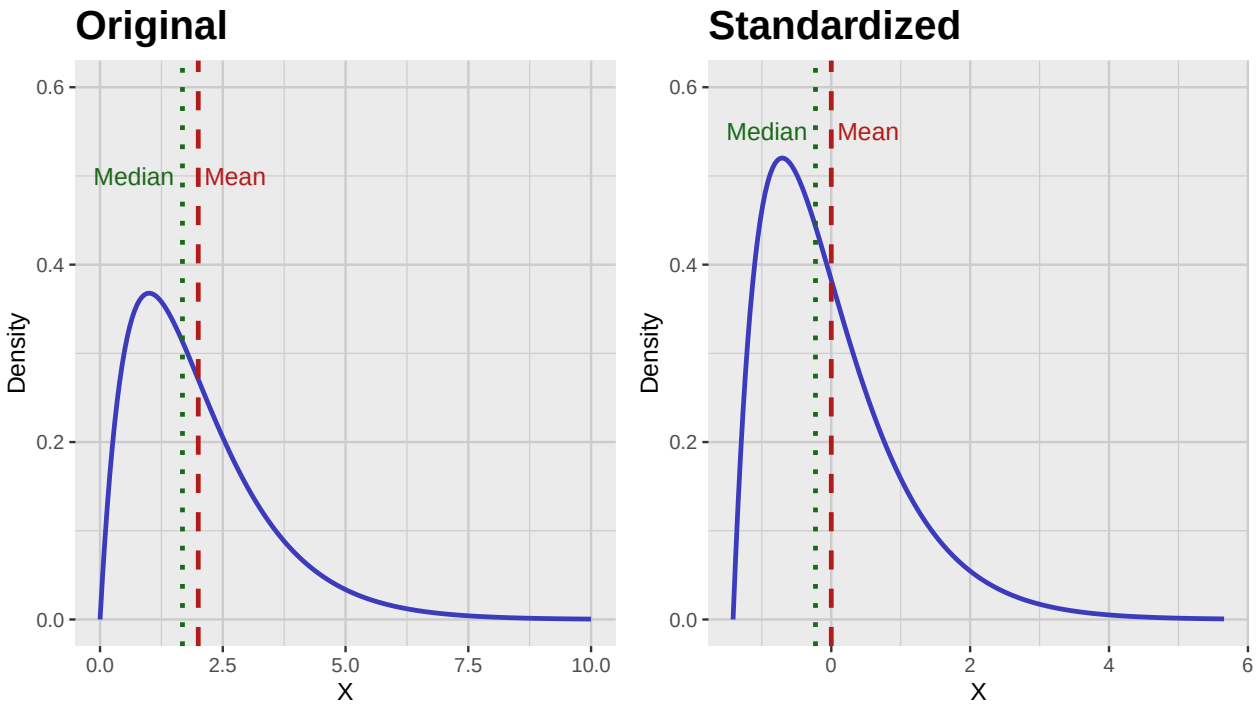

Figure 2.5: Comparison of the probability density function (PDF) of the original and standardized Gamma distribution.

2.5.2 Scaling Samples

For a given a sample, it is not always known what the underlying distribution is. In particular, we may not know what the mean and the standard deviation of the underlying population are. However, we can still try to scale our data but we should keep in mind that in this case there is more than one scaling method. The most common are:

1. *Standardization*: This method uses the sample mean and sample standard deviation to scale the data. The standardized sample is given by

$$T = \frac{X - \bar{X}}{S},$$

where $\bar{X}$ is the sample mean and S is the sample standard deviation.

2. *Min-Max Scaling*: This method scales the data to a fixed range, usually [0, 1]. The scaled sample is given by

$$T = \frac{X - \min(X)}{\max(X) - \min(X)},$$

where $\min(X)$ and $\max(X)$ are the minimum and maximum values of the sample, respectively. This method is very sensitive to outliers, as it uses the minimum and maximum values of the sample.

3. *Robust Scaling*: This method scales the data using the median and interquartile range (IQR). The scaled sample is given by
$$T = \frac{X - \text{median}(X)}{\text{IQR}(X)},$$
where $\text{IQR}(X) = Q_3 - Q_1$ is the interquartile range of the sample. This method is more robust to outliers than min-max scaling, as it uses the median and IQR instead of the minimum and maximum values.

4. *Normalization*: This method simply divides each data point by the corresponding vector norm. The normalized sample is given by
$$T = \frac{X}{\|X\|},$$
where $\|X\|$ is the vector norm of the sample. It is not used as often as the previous methods.

Remark 2.5.5. As we mentioned earlier, scaling does not make a non-normal distribution normal. In particular, the sample skewness and sample excess kurtosis of the scaled sample will be the same as those of the original sample.

As an example, we will consider the different scaling methods applied to the Gamma sample we generated earlier.

```
scale_lookup_tbl <- tibble(
  code = c(
    "sample",
    "sample_standard",
    "sample_min_max",
    "sample_robust",
    "sample_normal"
  ),
  label = c(
    "Original Sample",
    "Standardized",
    "Min-Max Scaled",
    "Robust Scaled",
    "Normalized"
  )
)

scaled_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    sample_standard = standard_scale(sample),
    sample_min_max = min_max_scale(sample),
    sample_robust = robust_scale(sample),
    sample_normal = normal_scale(sample)
  ) %>%
  pivot_longer(
    cols = everything(),
```

```
    names_to = "scaling_method",
    values_to = "sample_scaled"
  ) %>%
  convert_codes_to_factor(
    code_col = scaling_method,
    lookup_tbl = scale_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = scaling_method_factor
  )

scaled_gamma_qq_plot <- scaled_gamma_sample_tbl %>%
  ggplot(aes(sample = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plots",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

scaled_gamma_histogram <- scaled_gamma_sample_tbl %>%
  ggplot(aes(x = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
```

```
  ) +
  labs(
    title = "Histograms",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()

scaled_gamma_plot <- scaled_gamma_qq_plot + scaled_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Comparison of Scaling Methods Applied to the Gamma Sample",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

scaled_gamma_plot
```

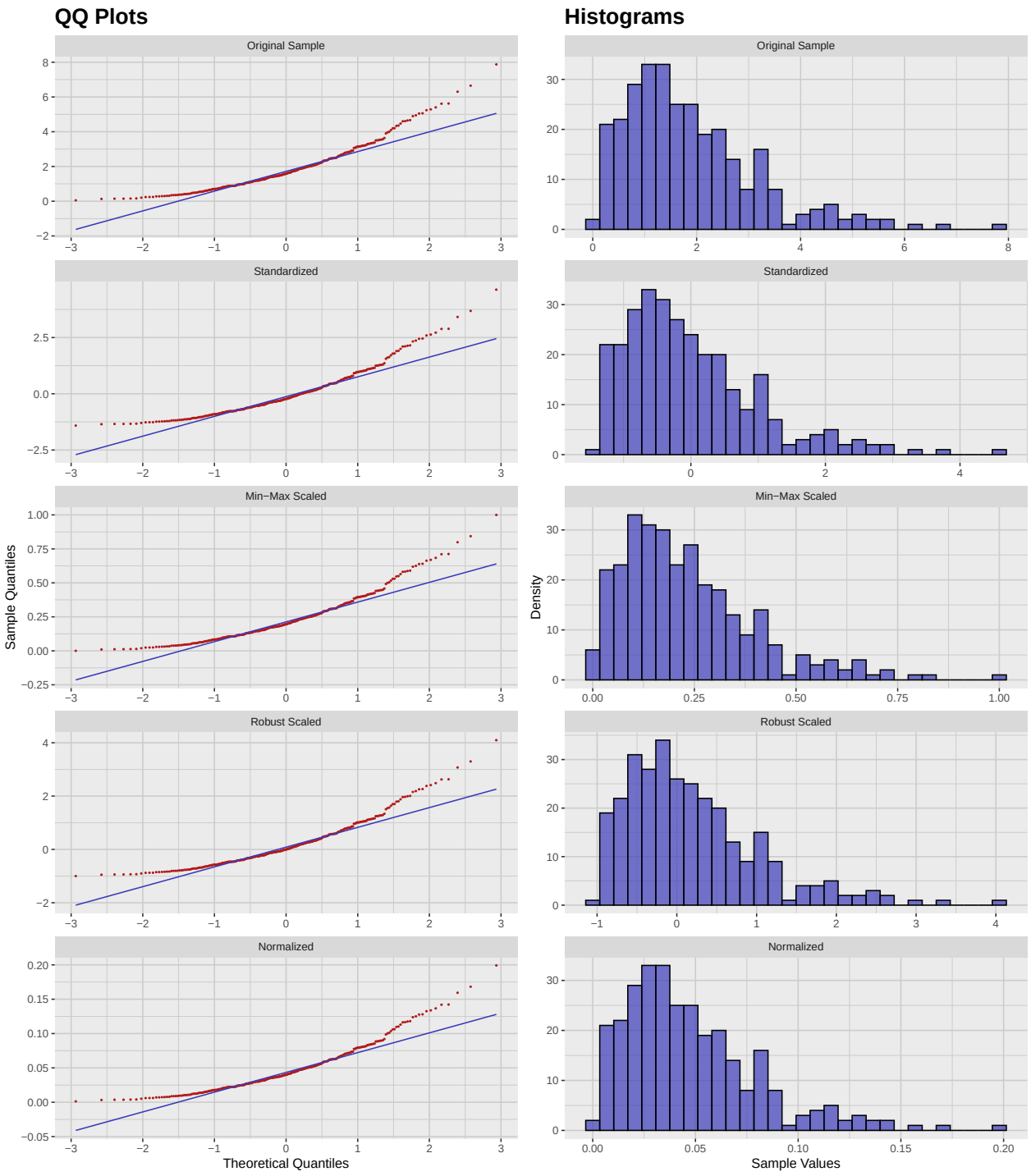

Figure 2.6: Comparison of different scaling methods applied to the Gamma distribution sample. Notice that the QQPlots are just rescaled versions of the original QQ plot, and therefore they all have the same shape.

2.6 Activity: Visualizing the Non-Normality of Samples

For each of the following distributions, construct a sample of size 300 and compute their associated sample Skewness and Kurtosis. After that, construct and compare their associated QQ plots and histograms.

1. Exponential distribution with rate parameter $\lambda = 1$.
2. Uniform distribution on the interval $[0, 1]$.
3. Beta distribution with shape parameters $\alpha = 2$ and $\beta = 5$.
4. Cauchy distribution with location parameter $x_0 = 0$ and scale parameter $\gamma = 1$.
5. Chi-squared distribution with degrees of freedom $k = 3$.

Chapter 3

Normality Testing

Although QQ plots are a good way to visually assess the normality of a dataset, they can be subjective and imprecise. Therefore, it becomes necessary to develop more objective and precise statistical tests to determine whether the data we are analyzing follows a normal distribution. In this chapter, we will explore some commonly used statistical tests for normality and discuss some transformations that can be performed on our data to make it more normal-like.

3.1 Normality Tests

There are several statistical tests that can be used to determine whether a dataset follows a normal distribution. Some of the most commonly used tests include:

1. *Shapiro-Wilk test*: This test is based on the correlation between the data and the corresponding normal scores. It is a powerful test for small sample sizes (less than 50) and is widely used in practice. However, it can be overly sensitive to small deviations from normality in larger sample sizes.

2. *Kolmogorov-Smirnov test*: This test compares the empirical distribution function of the data with the cumulative distribution function of the normal distribution. It is a non-parametric test and can be used for any sample size. However, it is less powerful than the Shapiro-Wilk test for small sample sizes. It is also sensitive to scale and location parameters, which can affect the results of the test.

3. *Anderson-Darling test*: This test is similar to the Kolmogorov-Smirnov test but gives more weight to the tails of the distribution. It is a powerful test for small sample sizes and is widely used in practice.

4. *Jarque-Bera test*: This test is based on the skewness and kurtosis of the data. It is a powerful test for large sample sizes and is widely used in practice. However, it can be overly sensitive to small deviations from normality in smaller sample sizes.

We will now apply these tests to the gamma sample data we created in the previous chapter:

```
normality_tests_lookup_tbl <- tibble(
  code = c("shapiro", "ks", "ad", "jb"),
  label = c(
    "Shapiro-Wilk",
    "Kolmogorov-Smirnov",
    "Anderson-Darling",
    "Jarque-Bera"
  ),
)

normality_test_gamma_sample_tbl <- gamma_sample_tbl %>%
  summarise(
    shapiro = tidy_shapiro_test(data = ., value_col = sample)$p_value,
    ks = tidy_ks_test(data = ., value_col = sample)$p_value,
    ad = tidy_ad_test(data = ., value_col = sample)$p_value,
    jb = tidy_jb_test(data = ., value_col = sample)$p_value
  ) %>%
  pivot_longer(
    everything(),
    names_to = "test",
    values_to = "p_value"
  ) %>%
  convert_codes_to_factor(
    code_col = test,
    lookup_tbl = normality_tests_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = test_factor
  ) %>%
  select(test_factor, p_value)
```

The results of the normality tests are as follows:

```
normality_test_gamma_sample_tbl
```

```
# A tibble: 4 x 2
  test_factor         p_value
  <fct>                 <dbl>
1 Shapiro-Wilk       1.58e-12
2 Kolmogorov-Smirnov 7.93e- 3
3 Anderson-Darling   9.02e-16
4 Jarque-Bera        0
```

As we can see, none of these tests was particularly successful, which is not surprising given that the gamma distribution is not normal.

3.2 Transformations

To make our data more normal-like, we can apply various transformations to any given sample. Some of the most common transformations include:

1. *Box-Cox transformation*: This is a family of power transformations that can be applied only to data that is always positive. It is given by the formula:

$$\tilde{X} = \begin{cases} \frac{X^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \log(X) & \text{if } \lambda = 0 \end{cases}$$

   where λ is some parameter.

2. *Yeo-Johnson transformation*: This transformation is similar to the Box-Cox transformation but can handle negative values. It is useful for data that is positively or negatively skewed. It is given by the formula:

$$\tilde{X} = \begin{cases} \frac{((X+1)^\lambda - 1)}{\lambda} & \text{if } X \geq 0, \lambda \neq 0 \\ \frac{-((-X+1)^{2-\lambda} - 1)}{2-\lambda} & \text{if } X < 0, \lambda \neq 2 \\ \log(X+1) & \text{if } X \geq 0, \lambda = 0 \\ \log(-X+1) & \text{if } X < 0, \lambda = 2 \end{cases}$$

   where λ is some parameter.

We will now apply the Box-Cox transformation to the gamma sample data with parameters $\lambda = 0$, 0.25, 0.5, 0.75, and 1:

```
box_cox_lookup_tbl <- tibble(
  code = c(
    "box_cox_0",
    "box_cox_025",
    "box_cox_05",
    "box_cox_075",
    "box_cox_1"
  ),
  label = c(
    "lambda = 0",
    "lambda = 0.25",
    "lambda = 0.5",
    "lambda = 0.75",
    "lambda = 1"
  ),
)

box_cox_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    box_cox_0 = bcPower(sample, lambda = 0),
    box_cox_025 = bcPower(sample, lambda = 0.25),
```

```
    box_cox_05 = bcPower(sample, lambda = 0.5),
    box_cox_075 = bcPower(sample, lambda = 0.75),
    box_cox_1 = bcPower(sample, lambda = 1)
  ) %>%
  pivot_longer(
    starts_with("box_cox_"),
    names_to = "lambda",
    values_to = "transformed_sample"
  ) %>%
  convert_codes_to_factor(
    code_col = lambda,
    lookup_tbl = box_cox_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = lambda_factor
  )


box_cox_gamma_qq_plot <- box_cox_gamma_sample_tbl %>%
  ggplot(aes(sample = transformed_sample)) +
  facet_wrap(
    ~lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plots",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

box_cox_gamma_histogram <- box_cox_gamma_sample_tbl %>%
  ggplot(aes(x = transformed_sample)) +
  facet_wrap(
    ~lambda_factor,
    nrow = 5,
```

```
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histograms",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()

box_cox_gamma_plot <- box_cox_gamma_qq_plot + box_cox_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Comparison of Box-Cox Transformations with Different Lambda Values",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

box_cox_gamma_plot
```

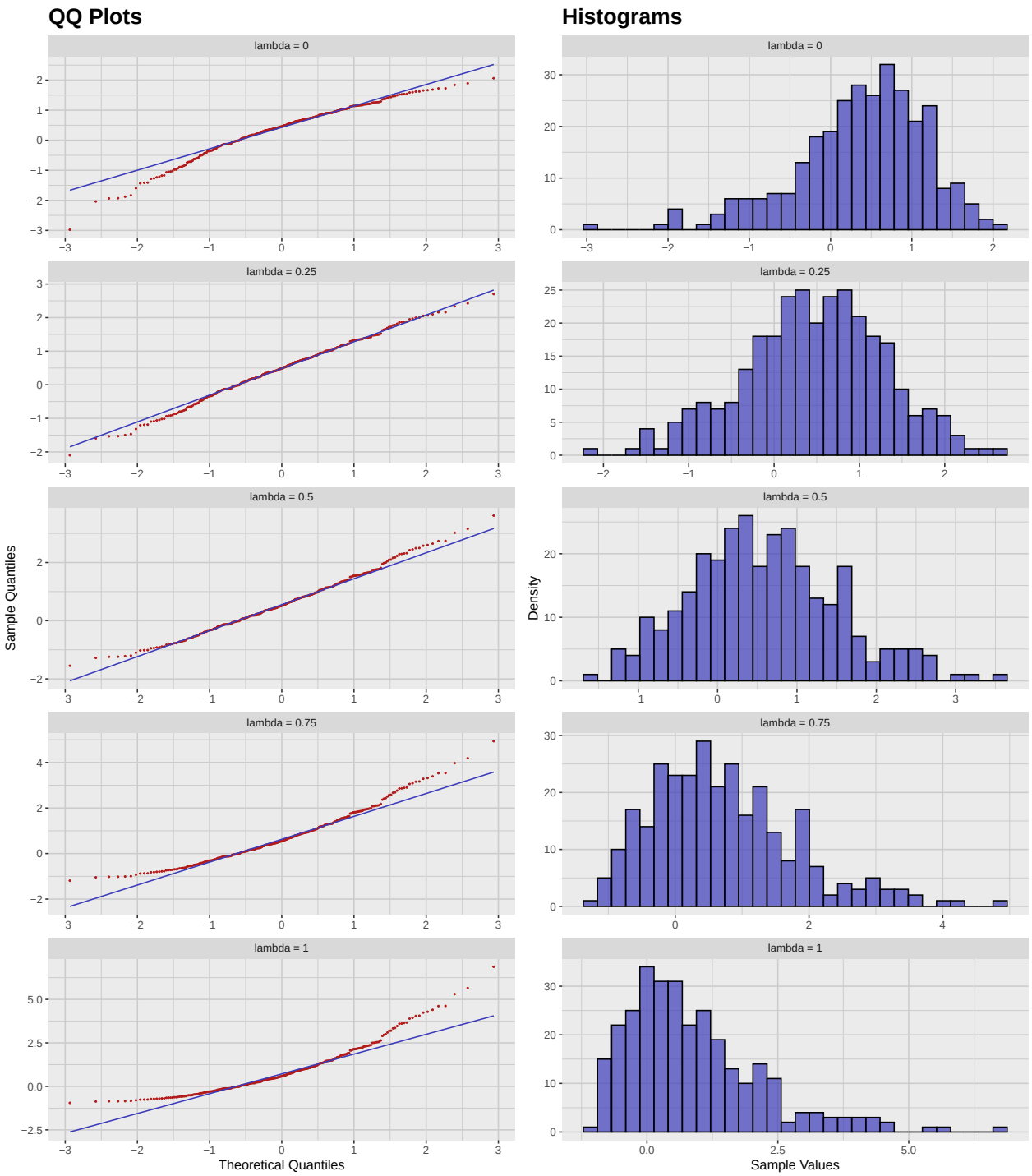

Figure 3.1: Comparison of different Box-Cox transformations applied to the Gamma distribution sample, with parameter lambda = 0, 0.25, 0.5, 0.75, and 1.

3.3 Normality Score Optimization

As we saw in the previous section, the Box-Cox transformation can be used to make our data more normal-like. However, the choice of the parameter λ is crucial for the success of the transformation. We wish to find the optimal value of λ that maximizes the log-likelihood of the transformed data. This can be done using the `powerTransform` function from the `car` package, which performs a Box-Cox transformation and estimates the optimal value of λ. The following figure shows the log-likelihood profile for the Box-Cox transformation applied to the gamma sample data:

```
# Intercept-only model
fit <- lm(sample ~ 1, data = gamma_sample_tbl)

# Estimate lambda
bc <- powerTransform(fit)

# Extract log-likelihood profile
bc_prof <- boxCox(fit, lambda = seq(-2, 2, 0.1), plotit = FALSE)

# Convert to tibble for ggplot
df <- tibble(lambda = bc_prof$x, loglik = bc_prof$y)

ggplot(df, aes(x = lambda, y = loglik)) +
  geom_line(color = c_pal("C blue"), linewidth = 1) +
  geom_vline(xintercept = bc$lambda, linetype = "dashed", color = c_pal("C red")) +
  labs(
    title = "Box-Cox Transformation",
    x = expression(lambda),
    y = "Log-likelihood"
  ) +
  theme_krul()
```

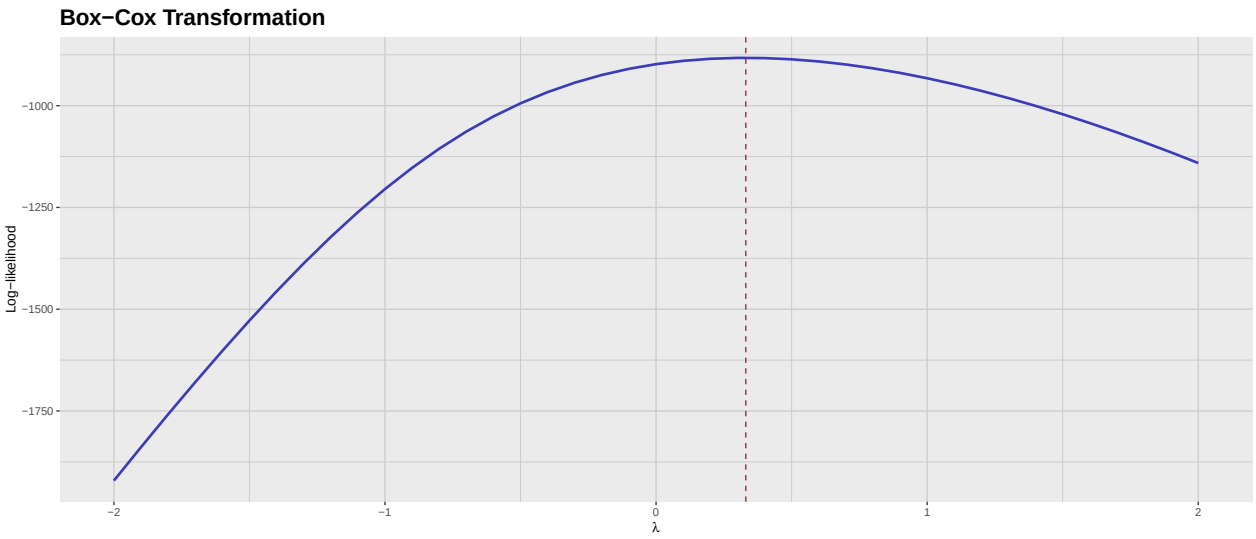

Figure 3.2: Log-likelihood profile for the Box-Cox transformation applied to the Gamma distribution sample. The dashed line indicates the optimal value of lambda.

Finally, we can extract the optimal value of λ and transform the data using this value:

```
optimal_lambda <- bc$lambda

optimal_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    transformed_sample = bcPower(sample, lambda = optimal_lambda)
  )

optimal_gamma_sample_tbl %>%
  summarise(
    shapiro = tidy_shapiro_test(data = ., value_col = transformed_sample)$p_value,
    ks = tidy_ks_test(data = ., value_col = transformed_sample)$p_value,
    ad = tidy_ad_test(data = ., value_col = transformed_sample)$p_value,
    jb = tidy_jb_test(data = ., value_col = transformed_sample)$p_value
  ) %>%
  pivot_longer(
    everything(),
    names_to = "test",
    values_to = "p_value"
  ) %>%
  convert_codes_to_factor(
    code_col = test,
    lookup_tbl = normality_tests_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = test_factor
```

```
  ) %>%
  select(test_factor, p_value)
```

```
# A tibble: 4 x 2
  test_factor        p_value
  <fct>                <dbl>
1 Shapiro-Wilk         0.974
2 Kolmogorov-Smirnov   1.000
3 Anderson-Darling     0.977
4 Jarque-Bera          0.878
```

As we can see, the Box-Cox transformation with the optimal value of λ significantly improves the normality of the data, as all the normality tests now yield high p-values. We now show the QQ plot and histogram of the transformed data:

```
optimal_gamma_qq_plot <- optimal_gamma_sample_tbl %>%
  ggplot(aes(sample = transformed_sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plot",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

optimal_gamma_histogram <- optimal_gamma_sample_tbl %>%
  ggplot(aes(x = transformed_sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogram",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()
```

```
optimal_gamma_plot <- optimal_gamma_qq_plot + optimal_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Optimal Gamma Distribution Transformation",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

optimal_gamma_plot
```

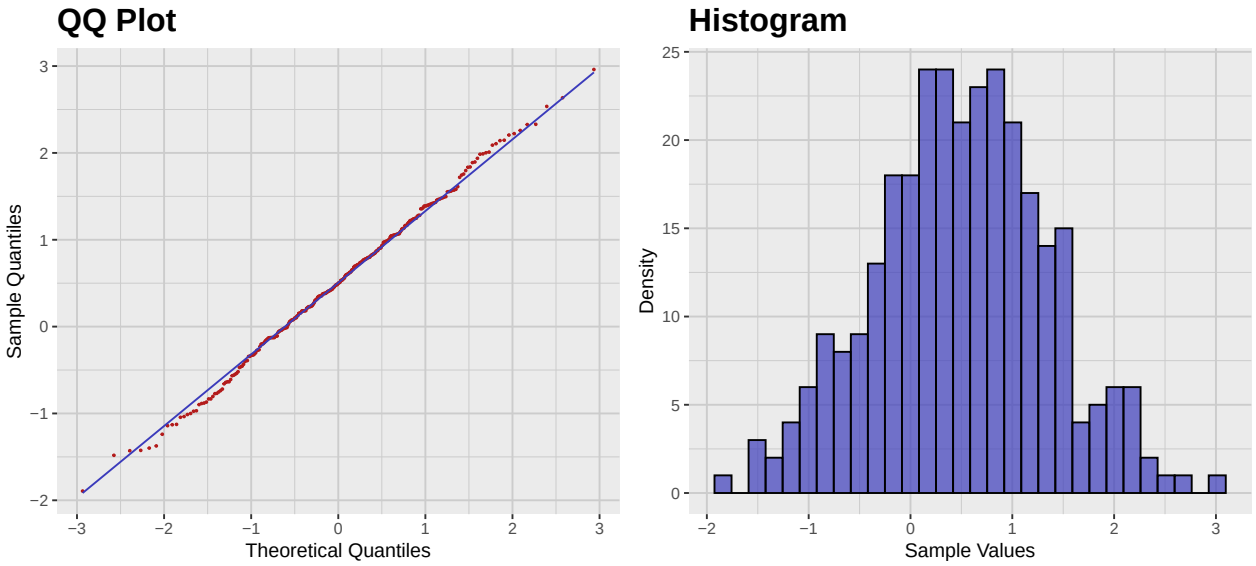

Figure 3.3: QQ plot and histogram of the Optimal Gamma Distribution Transformation. Notice how both plots show a much better fit to the normal distribution compared to the original gamma sample.

Part II

Principal Component Analysis

Chapter 4

Introduction to Multivariate Analysis

In this chapter we will introduce some of the most basic concepts in multivariate analysis, specifically focusing on the mean vector and covariance matrix of a random vector. These concepts are fundamental in understanding the relationships between multiple variables in a given dataset.

4.1 The Mean Vector and the Covariance Matrix

Definition 4.1.1. Let Π be a population and let

$$X : \Pi \to \mathbb{R}^p$$

be a random (row) vector. The *mean vector* of X is defined as

$$\mu = \mathbb{E}[X] = [\mathbb{E}[X_1], \mathbb{E}[X_2], \ldots, \mathbb{E}[X_p]] \tag{4.1}$$

where X_i is the i-th component of the random vector X. The *covariance matrix* of X is defined as

$$\Sigma = \mathrm{Cov}[X] = \mathbb{E}[(X-\mu)^T(X-\mu)] = [\mathrm{Cov}[X_i, X_j]]_{i,j=1}^p \tag{4.2}$$

where $\mathrm{Cov}[X_i, X_j] = \mathbb{E}[(X_i - \mathbb{E}[X_i])(X_j - \mathbb{E}[X_j])]$ is the covariance between the i-th and j-th components of the random vector X.

Remark 4.1.2. Remember that $\mathrm{Cov}[X_i, X_j] = \mathrm{Cov}[X_j, X_i]$, so the covariance matrix is symmetric. Furthermore, $\mathrm{Cov}[X_i, X_i] = \mathrm{Var}[X_i]$, which is the variance of the i-th component of the random vector X. For this reason, this matrix is also known as the *variance-covariance matrix* of X.

Now given a sample of size n taken from the population Π, its associated *sample matrix* is the $n \times p$ matrix where each row contains a single observation from the random vector X. In this context, the sample mean vector and the sample covariance matrix are defined as follows:

Definition 4.1.3. Given a sample of size n from the population Π, with associated sample matrix x, we define its associated *sample mean vector* as

$$\bar{x} = \frac{1}{n}\sum_{i=1}^n x_i = \left[\frac{1}{n}\sum_{i=1}^n x_{i,1}, \frac{1}{n}\sum_{i=1}^n x_{i,2}, \ldots, \frac{1}{n}\sum_{i=1}^n x_{i,p}\right] \tag{4.3}$$

where x_i is the i-th row of the sample matrix x and $x_{i,j}$ is the j-th component of the i-th row. The *sample covariance matrix* is defined as

$$\hat{\Sigma} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^T(x_i - \bar{x}) \tag{4.4}$$

where $\hat{\Sigma}$ is a $p \times p$ matrix and $\hat{\Sigma}_{i,j} = \frac{1}{n-1}\sum_{k=1}^{n}(x_{k,i} - \bar{x}_i)(x_{k,j} - \bar{x}_j)$ is the sample covariance between the i-th and j-th components of the random vector X.

Remark 4.1.4. The sample covariance matrix is an unbiased estimator of the covariance matrix of the random vector X.

Example 4.1.5. Consider the following sample matrix of size $n = 5$ from a population Π:

$$\begin{bmatrix} 2 & 5 & 7 \\ 4 & 3 & 6 \\ 6 & 4 & 5 \\ 8 & 2 & 4 \\ 10 & 1 & 3 \end{bmatrix}$$

To obtain the sample mean vector, we compute:

$$\bar{x}_1 = \frac{2+4+6+8+10}{5} = 6$$
$$\bar{x}_2 = \frac{5+3+4+2+1}{5} = 3$$
$$\bar{x}_3 = \frac{7+6+5+4+3}{5} = 5$$

In other words,

$$\bar{x} = [6, 3, 5]$$

The sample variance of X_j is:

$$\hat{\Sigma}_{X_j} = \frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x_j})^2$$

Therefore,

$$\hat{\Sigma}_{X_1} = \frac{(2-6)^2 + (4-6)^2 + (6-6)^2 + (8-6)^2 + (10-6)^2}{4} = 10$$
$$\hat{\Sigma}_{X_2} = \frac{(5-3)^2 + (3-3)^2 + (4-3)^2 + (2-3)^2 + (1-3)^2}{4} = 2.5$$
$$\hat{\Sigma}_{X_3} = \frac{(7-5)^2 + (6-5)^2 + (5-5)^2 + (4-5)^2 + (3-5)^2}{4} = 2.5$$

The sample covariance between X_j and X_k is:

$$\hat{\Sigma}_{X_j,X_k} = \frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

Calculating the sample covariances, we have:

$$\begin{aligned}
\hat{\Sigma}_{X_1,X_2} &= \frac{(2-6)(5-3)+(4-6)(3-3)+(6-6)(4-3)+(8-6)(2-3)+(10-6)(1-3)}{4} \\
&= \frac{(-8)+0+0+(-2)+(-8)}{4} = -4.5 \\
\hat{\Sigma}_{X_1,X_3} &= \frac{(2-6)(7-5)+(4-6)(6-5)+(6-6)(5-5)+(8-6)(4-5)+(10-6)(3-5)}{4} \\
&= \frac{(-8)+(-2)+0+(-2)+(-8)}{4} = -5 \\
\hat{\Sigma}_{X_2,X_3} &= \frac{(5-3)(7-5)+(3-3)(6-5)+(4-3)(5-5)+(2-3)(4-5)+(1-3)(3-5)}{4} \\
&= \frac{4+0+0+1+4}{4} = 2.25
\end{aligned}$$

The resulting covariance matrix is then:

$$\hat{\Sigma} = \begin{bmatrix} 10 & -4.5 & -5 \\ -4.5 & 2.5 & 2.25 \\ -5 & 2.25 & 2.5 \end{bmatrix}$$

4.2 The Mahalanobis Distance

Definition 4.2.1. Assume that we have a population Π with a random vector X that has mean vector μ and covariance matrix Σ. If Σ is invertible, then the *Mahalanobis distance* between two points x and y in $\mathbb{R}^p$ is defined as:

$$d_M(x,y) = \sqrt{(x-y)\Sigma^{-1}(x-y)^T} \tag{4.5}$$

In the context of sample data, the Mahalanobis distance can be computed using the sample covariance matrix $\hat{\Sigma}$:

Definition 4.2.2. Given a sample of size n from the population Π, with associated sample matrix x and sample covariance matrix $\hat{\Sigma}$, the *sample Mahalanobis distance* between two points x_i and x_j is defined as:

$$d_M(x_i,x_j) = \sqrt{(x_i-x_j)\hat{\Sigma}^{-1}(x_i-x_j)^T} \tag{4.6}$$

Remark 4.2.3. The Mahalanobis distance is a measure of the distance between two points in a multivariate space, taking into account the correlations between the variables. It is particularly useful for identifying outliers in multivariate data.

To illustrate this point, we will consider the following dataset: We start by taking a random sample of size $n = 300$ from a uniform distribution in the interval $[0, 10]$.

```
set.seed(123)
n <- 300
x_data <- runif(n, min = 0, max = 10)
```

Now we will construct some error vector ϵ from a normal distribution with mean 0 and standard deviation 1:

```
epsilon_data <- rnorm(n, mean = 0, sd = 1)
```

We can now construct the vector

```
y_data <- x_data + epsilon_data
```

and put it together to create a tibble:

```
sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

The associated scatter plot of the data is shown below:

```
sample_data_plot <- sample_data_tbl %>%
  ggplot(aes(x = x_data, y = y_data)) +
  geom_point(
    color = "black",
    size = 0.2,
    alpha = 1
  ) +
  labs(
    title = "Sample Data",
    x = "X",
    y = "Y"
  ) +
  coord_fixed() +
  theme_krul()

sample_data_plot
```

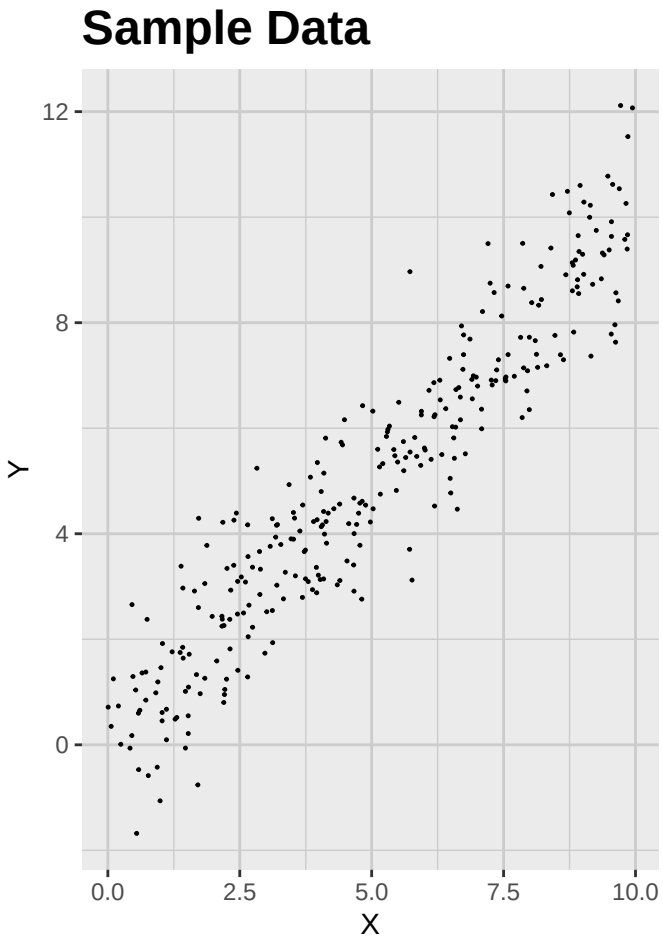

Figure 4.1: Scatter plot of the sample data. Notice that the X and Y variables are highly correlated.

We now add two extra points to this plot:

```
extra_points_tbl <- tibble(
  x_extra = c(1, 5),
  y_extra = c(1, 2)
)

extra_sample_data_plot <- sample_data_plot +
  geom_point(
    data = extra_points_tbl,
    aes(x = x_extra, y = y_extra),
    color = c_pal("C red"),
    size = 1.5
  )

extra_sample_data_plot
```

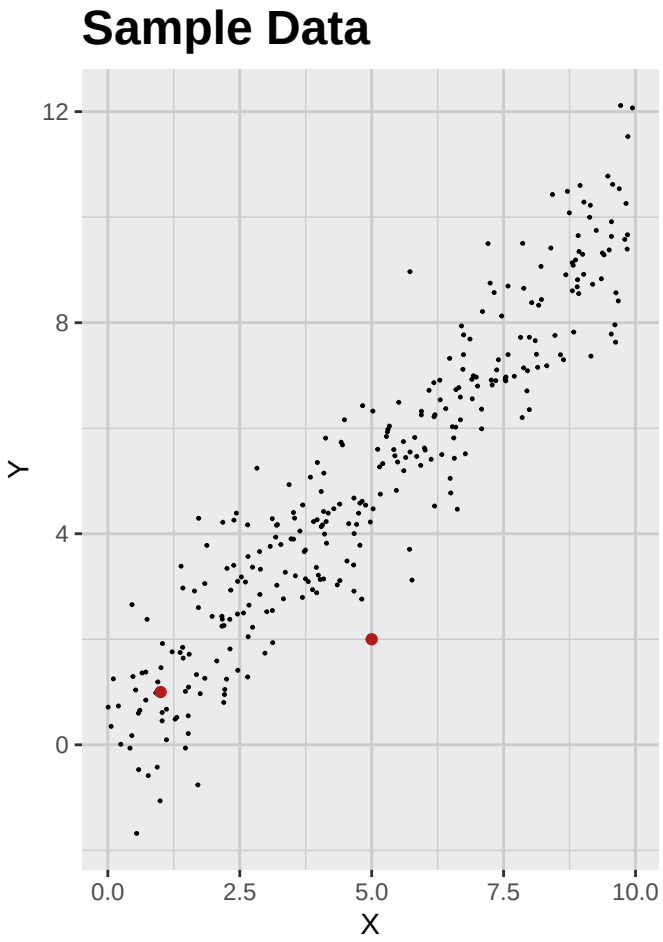

Figure 4.2: Scatter Plot with extra sample points. Notice that one of these points is clearly an outlier.

Notice that one of these points is clearly an outlier. We would like to have some concrete measure to show this, so we start by computing the sample mean vector which represents the center or our data. For future reference, we will also compute the sample covariance matrix.

```
sample_mean_vector <- colMeans(sample_data_tbl)
sample_covariance_matrix <- cov(sample_data_tbl)

sample_mean_tbl <- tibble(
  x_bar = sample_mean_vector[1],
  y_bar = sample_mean_vector[2]
)
```

But now if we use the Euclidean distance, we obtain that it is actually the outlier point that is closer to the sample mean vector:

```
euclidean_sample_data_plot <- extra_sample_data_plot +
  geom_point(
```

```
    data = sample_mean_tbl,
    aes(x = x_bar, y = y_bar),
    color = c_pal("C red"),
    size = 1.5
  ) +
  geom_circle(
    aes(x0 = sample_mean_vector[1], y0 = sample_mean_vector[2], r = 4),
    color = c_pal("C blue"),
    linewidth = 0.6,
    fill = NA
  )

euclidean_sample_data_plot
```

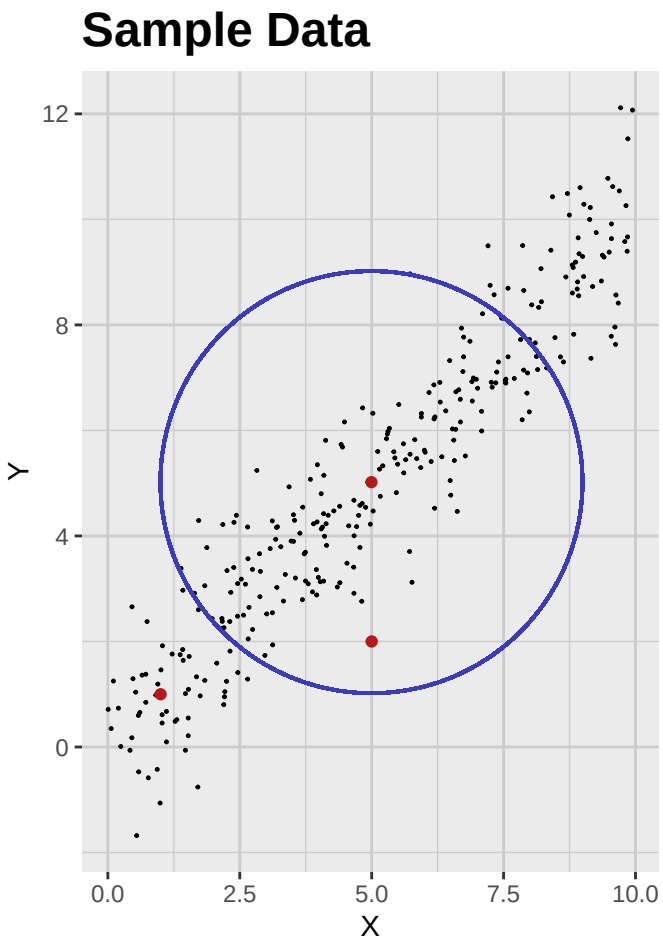

Figure 4.3: Scatter Plot with sample mean vector and a circle representing points that are at a constant Euclidean distance from the center of our data.

On the other hand, if we compute the Mahalanobis distance, we obtain that the outlier point is

actually further away from the sample mean vector:

```
annotated_sample_data_plot <- extra_sample_data_plot +
  geom_point(
    data = sample_mean_tbl,
    aes(x = x_bar, y = y_bar),
    color = c_pal("C red"),
    size = 1.5
  ) +
  stat_ellipse(
    type = "norm",
    level = 0.8647,
    color = c_pal("C blue"),
    linewidth = 0.6
  )

annotated_sample_data_plot
```

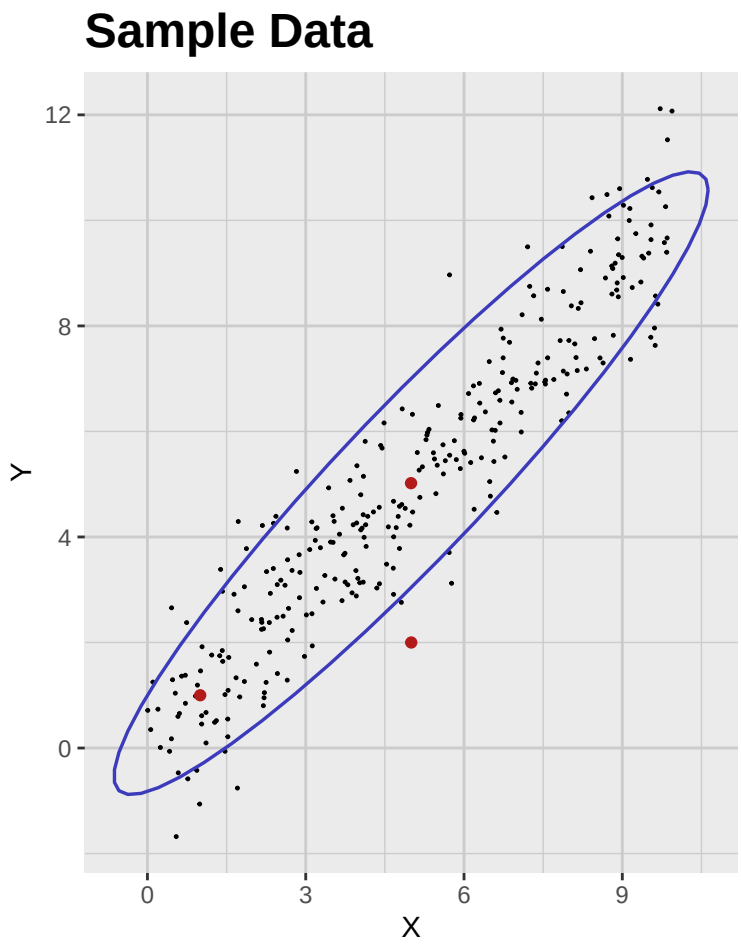

Figure 4.4: Scatter Plot with sample mean vector and a ellipse representing points that are at a constant Mahalanobis distance from the center of our data.

4.3 Scaling and Standardization

Now assume that we have a random vector X with mean vector μ and covariance matrix Σ. We can define a new random vector Y as follows:

$$Y^T = AX^T + b$$

where A is a $p \times p$ matrix and b is a p-dimensional vector. The mean vector and covariance matrix of the new random vector Y can be computed as follows:

$$\mu_Y^T = \mathbb{E}[Y]^T = A\mu^T + b$$

and

$$\Sigma_Y = \text{Cov}[Y] = A\Sigma A^T$$

As we will show later, it is always possible to find a matrix A and a vector b such that the new random vector Y has mean vector $\mu_Y = 0$ and covariance matrix $\Sigma_Y = I_p$, where I_p is the identity matrix of size $p \times p$. This process is known as a *standardization* of the random vector X.

4.4 Activity: The Geometry of Multivariate Data

Part A: Theoretical Exercises

1. Prove that the Mahalanobis distance reduces to the Euclidean distance if the covariance matrix is the identity.
2. Show that Mahalanobis distance is invariant under affine transformations.
3. Show that if you standardize each variable in a multivariate dataset, the covariance matrix becomes the coorrelation matrix.
4. Assume that X_1 and X_2 are two independent random variables following the standard normal distribution. Show that $d_M(X, 0)^2$ follows a chi-square distribution with 2 degrees of freedom, where $d_M(X, 0)$ is the Mahalanobis distance from the origin.

Part B: Practical Exercises

1. Generate a 2D dataset of 50 points (two correlated variables).
2. Compute the sample mean vector and covariance matrix.
3. Write a function in R that computes the Mahalanobis distance of each point from the mean.
4. Identify the point with the largest Mahalanobis distance.
5. Plot the dataset using `ggplot2`.
6. Draw Mahalanobis distance ellipses for distances 1, 2, and 3.
7. Color points depending on whether their distance is greater than 1, 2, or 3.
8. Create a 3-variable dataset.
9. Compute Mahalanobis distances in 3D.

10. Plot pairs of variables with projected ellipses.
11. Compare Euclidean distances from the mean with Mahalanobis distances.

Part C: Advanced Exercises

1. Simulate 5 points in 3 dimensions with one variable as a linear combination of the others.
2. Try to compute Mahalanobis distance and discuss singularity of covariance.
3. Apply a linear transformation to a 2D dataset and verify invariance of Mahalanobis distances.
4. Write an R function that returns points forming a Mahalanobis-distance ellipse.
5. Overlay Mahalanobis-distance ellipses of arbitrary radius on a scatter plot.

Chapter 5

Normal Multivariate Distribution

5.1 Definition and Basic Properties

Definition 5.1.1. Let μ be a (row) vector and let Σ be a symmetric positive definite matrix. We say that a random (row) vector X has *normal multivariate distribution* with mean μ and covariance matrix Σ, in symbols $X \sim \mathcal{N}_p(\mu, \Sigma)$, if its probability density is given by

$$\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(X-\mu)\Sigma^{-1}(X-\mu)^T\right) dX$$

Remark 5.1.2. Some things to notice about the definition above:

1. Just like in the univariate case, the normal multivariate distribution is completely determined by its mean μ and covariance matrix Σ.
2. The covariance matrix Σ must be symmetric and positive definite, which ensures that the distribution is well-defined and that the exponent in the density function is negative definite.
3. The term
$$|\Sigma| = \det(\Sigma)$$
is the *generalized variance* of the distribution. It can be interpreted as a measure of the "spread" of the distribution in the p-dimensional space.
4. Notice that the term $(X-\mu)\Sigma^{-1}(X-\mu)^T$ is the square of the Mahalanobis distance between the random vector X and the mean μ.

Just like in the univariate case, the normal multivariate distribution is singled out among all other possible distributions by the Central Limit Theorem, that roughly states that the sum of a large number of independent and identically distributed random variables (properly standardized) converges to a normal distribution. The precise statement is as follows:

Theorem 5.1.3 (Multivariate Central Limit Theorem)**.** *Let $\{X_i\}_{i=1}^{\infty}$ be a sequence of independent and identically distributed random vectors in $\mathbb{R}^p$ with mean vector μ and covariance matrix Σ. Define*

the sample mean

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

Then, as $n \to \infty$,

$$\sqrt{n}\,(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}_p(0, \Sigma),$$

where $\xrightarrow{d}$ *denotes convergence in distribution and* $\mathcal{N}_p(0, \Sigma)$ *is the p-dimensional multivariate normal distribution with mean vector* 0 *and covariance matrix* Σ.

Remark 5.1.4. Even though the Central Limit Theorem singles out the normal multivariate distribution as a particularly important distribution, it is not the only relevant distribution in multivariate statistics. Actually, most real-world data does not follow a normal distribution, and many statistical methods are designed to handle non-normal data. Furthermore, important statistical methods, like linear regression, principal component analysis and clustering do not yield good results when applied to a normal multivariate distribution.

5.2 Normal Multivariate Contours

Notice that the probability density function of the normal multivariate distribution is a Gaussian function in p dimensions. This means that the contours of the distribution are ellipsoids centered at the mean μ and whose shape is controlled by Σ. The probability distribution of these contours is controlled by the χ^2 distribution in the following sense: let $0 \leq \varpi \leq 1$ be a probability level, and let c be such that

$$\mathbb{P}(\chi^2_p \leq c) = \varpi$$

Then, the contour that contains a proportion ϖ of the probability mass of the distribution is given by the set of points X that satisfy the equation

$$(X - \mu)\Sigma^{-1}(X - \mu)^T = c$$

In the following figure, we show the contours of the bivariate normal distribution with different covariance matrices. The contours are ellipses centered at the mean $\mu = (0, 0)$, and their shape is determined by the covariance matrix Σ. The color gradient represents the density of the distribution, with darker colors indicating higher density.

```r
# Add seed for reproducibility
set.seed(123)

# Parameters
mu <- c(0, 0)
Sigma_1 <- matrix(c(1, 0, 0, 1), nrow = 2)
Sigma_2 <- matrix(c(0.3, 0, 0, 1.5), nrow = 2)
Sigma_3 <- matrix(c(1, 0.6, 0.6, 1), nrow = 2)
Sigma_4 <- matrix(c(1, -0.6, -0.6, 1), nrow = 2)
```

```r
# Function to generate contour + scatter plot
plot_bivariate_normal <- function(mu, Sigma, n_samples = 1000,
                                  xlim = c(-3, 3), ylim = c(-3, 3),
                                  grid_length = 100) {
  # Create grid
  x <- seq(xlim[1], xlim[2], length.out = grid_length)
  y <- seq(ylim[1], ylim[2], length.out = grid_length)
  grid <- expand.grid(x = x, y = y)
  grid$z <- dmvnorm(grid, mean = mu, sigma = Sigma)

  # Sample points
  samples <- rmvnorm(n_samples, mean = mu, sigma = Sigma)
  colnames(samples) <- c("x", "y")
  samples <- as_tibble(samples)

  # Plot
  normal_plot <- ggplot() +
    geom_contour(data = grid, aes(x, y, z = z, color = after_stat(level))) +
    geom_point(
      data = samples,
      aes(x, y),
      color = c_pal("C red"),
      alpha = 0.5,
      size = 0.1
    ) +
    scale_color_viridis_c(option = "mako") +
    coord_cartesian(xlim = xlim, ylim = ylim) +
    labs(
      title = "",
      x = expression(X[1]), y = expression(X[2]),
      color = "Density"
    ) +
    theme_krul()

  return(normal_plot)
}

normal_plot_1 <- plot_bivariate_normal(mu, Sigma_1)
normal_plot_2 <- plot_bivariate_normal(mu, Sigma_2)
normal_plot_3 <- plot_bivariate_normal(mu, Sigma_3)
normal_plot_4 <- plot_bivariate_normal(mu, Sigma_4)

normal_plot <- (normal_plot_1 + normal_plot_2) /
  (normal_plot_3 + normal_plot_4) +
  plot_layout(widths = c(1, 1), heights = c(1, 1)) +
```

```
  plot_annotation(
    title = "Contours of the Bivariate Normal ",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

normal_plot
```

Contours of the Bivariate Normal

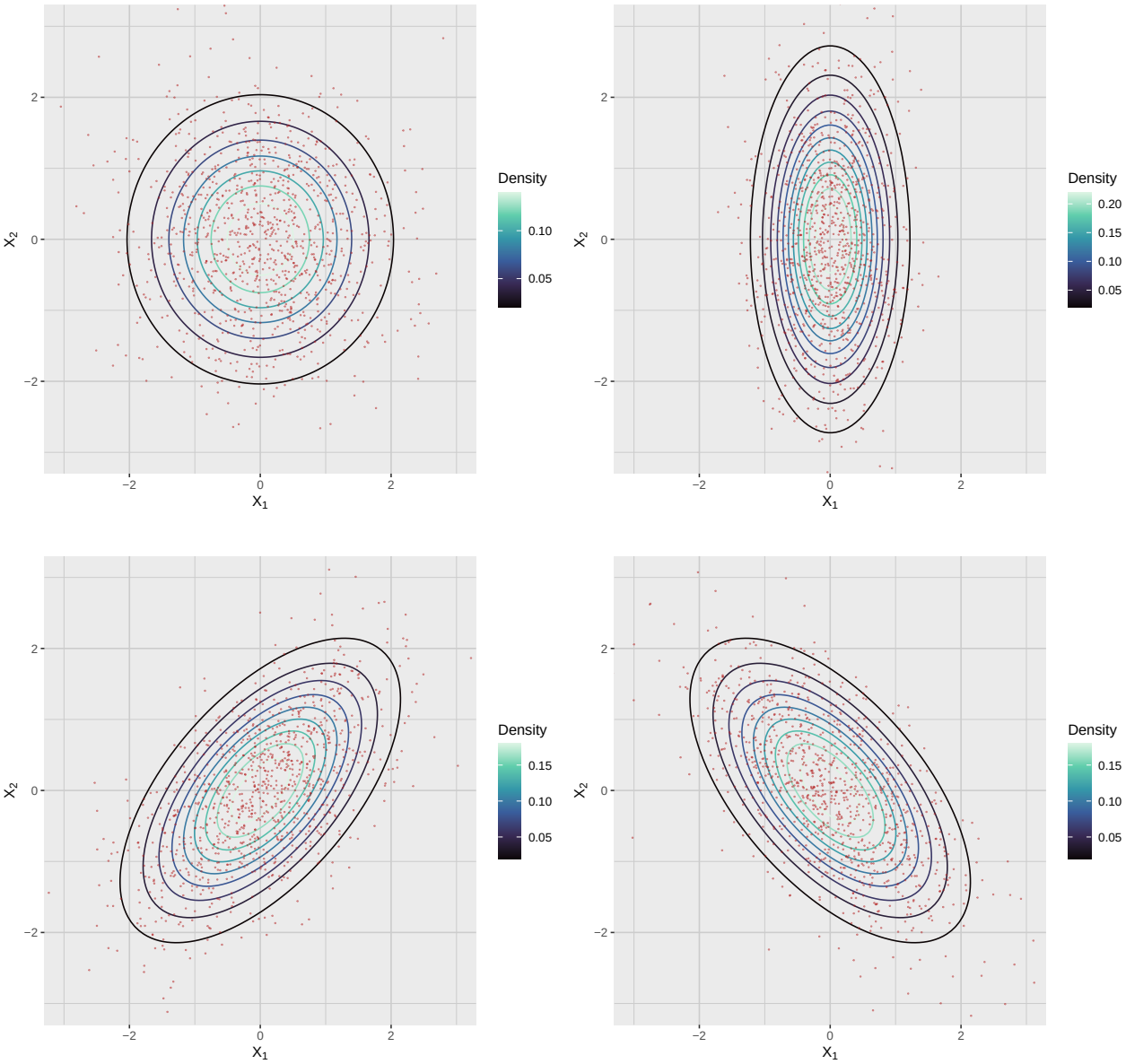

Figure 5.1: Bivariate Normal Distribution with Different Covariance Matrices

5.3 Standardization of the Normal Multivariate Distribution

Just like in the univariate case, given a random vector $X \sim \mathcal{N}_p(\mu, \Sigma)$, we can standardize it to obtain a new random vector Z that follows the standard multivariate normal distribution $\mathcal{N}_p(0, I_p)$, where I_p is the identity matrix of size p. To describe this standardization process, we first need to remember some linear algebra results.

Theorem 5.3.1 (Singular Value Decomposition)**.** *Let Σ be a symmetric positive definite matrix of*

size $p \times p$. Then, there exists an orthogonal matrix Q and a diagonal matrix Λ with positive entries such that

$$\Sigma = Q\Lambda Q^T$$

Note 5.3.2. Remember that a matrix Q is orthogonal if $Q^TQ = I_p$, where I_p is the identity matrix of size p. This means that the columns of Q are orthonormal vectors, and thus they form a basis for $\mathbb{R}^p$.

Remark 5.3.3. The singular value decomposition (SVD) is a powerful tool in linear algebra that allows us to decompose a matrix into its constituent parts. In the case of the covariance matrix Σ, the SVD provides a way to express it in terms of its *singular values* (the diagonal entries of Λ, also known as its *eigenvalues*) and its *singular vectors* (the columns of Q, also known as its *eigenvectors*). This decomposition is particularly useful for understanding the geometry of the multivariate normal distribution.

Notice that, since Σ is positive definite, the diagonal entries of

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_p \end{bmatrix}$$

are all positive. In particular, we can define a new matrix

$$D = \begin{bmatrix} \sqrt{\lambda_1} & 0 & \cdots & 0 \\ 0 & \sqrt{\lambda_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sqrt{\lambda_p} \end{bmatrix}$$

satisfying $D^2 = \Lambda$. If we now take

$$\sigma = DQ^T$$

then

$$\Sigma = \sigma^T\sigma$$

In particular, we can define the *standardization* of the random vector X as

$$Z = (X - \mu)\sigma^{-1} = (X - \mu)QD^{-1} = (X - \mu)Q\Lambda^{-1/2}$$

in which case $Z \sim \mathcal{N}_p(0, I_p)$.

5.4 Geometric Interpretation of the SVD

Earlier in this chapter we considered the Normal Multivariate Distribution with mean

$$\mu = [0, 0]$$

and covariance matrix

$$\Sigma = \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix}$$

According to the SVD, we can write this covariance matrix as

$$\Sigma = Q\Lambda Q^T$$

for some orthogonal matrix Q and diagonal matrix Λ. In this case, we can compute the SVD of Σ as follows:

```
# Example mean and covariance
mu <- c(0, 0)
Sigma <- matrix(c(
  1, 0.6,
  0.6, 1
), 2, 2)


# Compute eigenvectors and eigenvalues
eig <- eigen(Sigma)
vectors <- eig$vectors
values <- eig$values
```

We obtain:

$$Q = \begin{bmatrix} 0.7071068 & -0.7071068 \\ 0.7071068 & 0.7071068 \end{bmatrix}$$

and

$$\Lambda = \begin{bmatrix} 1.6 & 0 \\ 0 & 0.4 \end{bmatrix}$$

where the singular vectors are the columns of Q and the singular values are the diagonal entries of Λ. Geometrically, the singular vectors represent the directions of the axes of the ellipsoid that defines the contours of the normal multivariate distribution, and the square root of the singular values represent the lengths of these axes.

```
# Scale eigenvectors by sqrt(eigenvalues) for plotting
vec_data <- tibble(
  x0 = mu[1],
  y0 = mu[2],
  x1 = mu[1] + sqrt(values) * vectors[1, ],
  y1 = mu[2] + sqrt(values) * vectors[2, ]
)

# Generate points of the 1-Mahalanobis-distance ellipse
ellipse_points <- as_tibble(ellipse(Sigma, centre = mu, level = 0.393))
# level = 0.393 corresponds to ~1 standard deviation for 2D


# Plot
ggplot() +
  geom_path(
```

```
  data = ellipse_points,
  aes(x = x, y = y),
  color = c_pal("C blue"),
  linewidth = 1
) +
geom_segment(
  data = vec_data,
  aes(x = x0, y = y0, xend = x1, yend = y1),
  arrow = arrow(length = unit(0.2, "cm")),
  color = c_pal("C green"),
  linewidth = 1
) +
geom_point(
  aes(x = mu[1], y = mu[2]),
  color = c_pal("C red"),
  size = 3
) +
coord_equal() +
labs(
  title = "Geometric Interpretation of the SVD",
  x = expression(X[1]),
  y = expression(X[2])
) +
theme_krul()
```

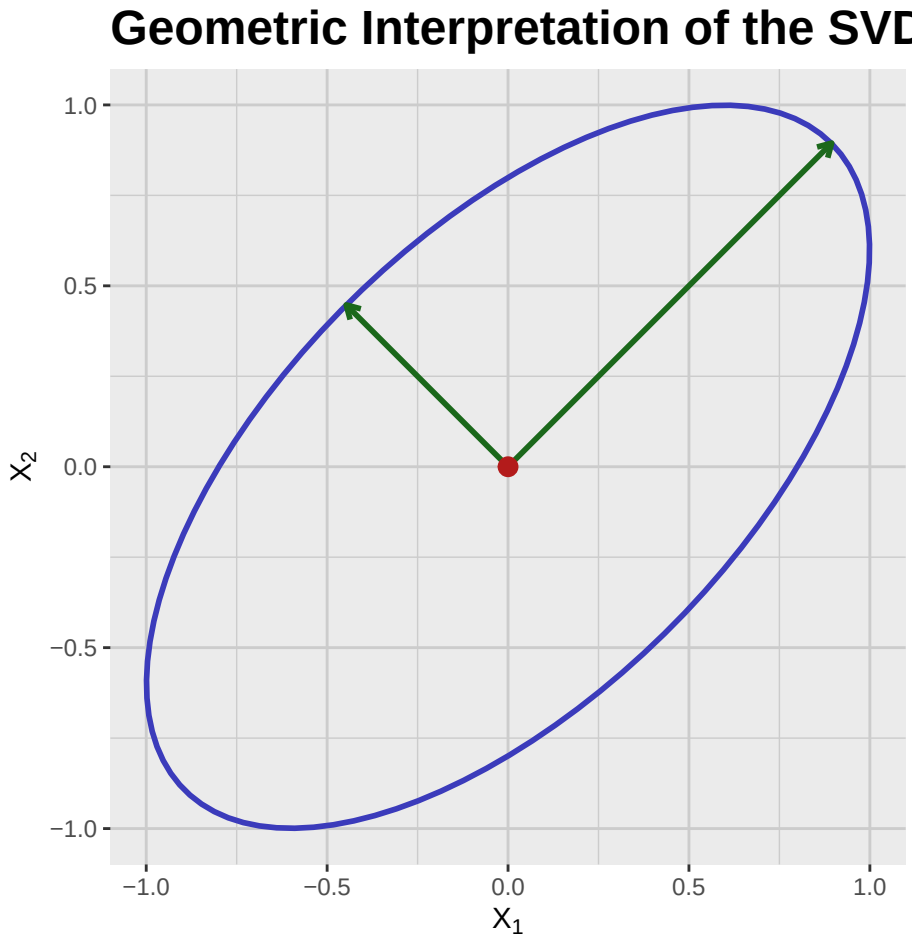

Figure 5.2: Geometric Interpretation of the SVD of a Bivariate Normal Distribution. The red point represents the mean, the green arrows represent the eigenvectors scaled by the square root of the eigenvalues, and the blue ellipse represents the 1-Mahalanobis-distance contour.

5.5 Closure under Linear Transformations

Theorem 5.5.1 (Closure under Linear Transformations)**.** *Let $X \sim \mathcal{N}_p(\mu, \Sigma)$ be a random vector with normal multivariate distribution. Then, for any matrix A of size $m \times p$ and any vector b of size m, the random vector*

$$Y = AX + b$$

also has a normal multivariate distribution, specifically $Y \sim \mathcal{N}_m(A\mu + b, A\Sigma A^T)$.

Remark 5.5.2. This theorem states that if we apply a linear transformation to a random vector with a normal multivariate distribution, the resulting random vector will also have a normal multivariate distribution. The mean and covariance of the transformed vector are given by the formulas in the theorem. This property is particularly useful in applications such as regression analysis, where we often need to transform the data to fit a model while maintaining the normality of the residuals. Furthermore, it allows us to derive new distributions from existing ones, which is a key aspect of

multivariate statistical theory. In practice, this theorem is often used to derive the distribution of linear combinations of random variables, which is a common operation in multivariate statistics.

Example 5.5.3. Let $X \sim \mathcal{N}_2(\mu, \Sigma)$ be a random vector with mean $\mu = (1, 2)$ and covariance matrix

$$\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix}$$

Now, consider the linear transformation

$$Y = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} X + \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

In other words,

$$\begin{aligned} Y_1 &= 2X_1 + X_2 \\ Y_2 &= 3X_2 + 1 \end{aligned}$$

Using the closure under linear transformations theorem, we can find the mean and covariance of Y:

$$\begin{aligned} \text{Mean: } A\mu + b &= \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \end{bmatrix} \\ \text{Covariance: } A\Sigma A^T &= \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 8 & 9 \\ 9 & 18 \end{bmatrix} \end{aligned}$$

Thus,

$$Y \sim \mathcal{N}_2 \left(\begin{bmatrix} 4 \\ 7 \end{bmatrix}, \begin{bmatrix} 8 & 9 \\ 9 & 18 \end{bmatrix} \right)$$

Remark 5.5.4. Notice that since projection into a subspace is a linear transformation, the closure under linear transformations theorem implies that any subset of the components of a normal multivariate distribution also has a normal multivariate distribution. This is a very useful property, as it allows us to analyze subsets of variables while still maintaining the normality assumption.

5.6 Multivariate Normality Testing

There are several statistical tests to check whether a given dataset follows a multivariate normal distribution. These tests are essential in multivariate statistics, as many methods assume normality. Here are some of the most commonly used tests, together with their implementation in R:

1. **Mardia's Test:** Based on multivariate skewness and kurtosis. For a multivariate normal distribution, skewness should be close to 0 and kurtosis close to $p(p+2)$, where p is the number of variables. Implemented in R via `MVN::mvn(your_data, mvn_test = "mardia")`.
2. **Henze-Zirkler (HZ) Test:** Uses the empirical characteristic function to measure deviations from normality. It has good global power and is suitable for moderate dimensions. Implemented in R via `MVN::mvn(your_data, mvn_test = "hz")`.

3. **Royston's Test:** A multivariate extension of the Shapiro-Wilk test, based on the product of univariate Shapiro-Wilk statistics applied to principal components. Easy to interpret, though power decreases in high dimensions. Implemented in R via `MVN::mvn(your_data, mvn_test = "royston")`.

4. **Doornik-Hansen Test:** Combines skewness and kurtosis into a single omnibus test, providing a robust alternative to Mardia. Implemented in R via `MVN::mvn(your_data, mvn_test = "dh")`.

5. **Graphical Methods (Chi-square Q-Q plots):** Plot the Mahalanobis distances of the data against the quantiles of a chi-square distribution. Deviations from a straight line indicate departures from multivariate normality. Implemented in R via `multivariate_diagnostic_plot(data = your_data, type = "qq")` .

Chapter 6

Principal Component Analysis

Principal Component Analysis (PCA) is a powerful technique used to reduce the dimensionality of a dataset while preserving as much information (measured by variance) as possible. It transforms the original variables into a new set of uncorrelated variables called principal components, which are ordered by the amount of variance they explain.

6.1 Understanding Total Variance

Let Σ be the covariance matrix of a given dataset. The *total variance* of the data is given by the trace of the covariance matrix:

$$\text{Total Variance} = \operatorname{Tr}\Sigma = \sum_{i=1}^{p} \Sigma_{ii},$$

where Σ_{ii} are the diagonal elements of Σ representing the sample variance of each variable. Now, using the singular value decomposition (SVD) of Σ, we can find matrices Q, and Λ such that:

$$\Sigma = Q\Lambda Q^T,$$

where Λ is a diagonal matrix containing the singular values $\lambda_1, \ldots, \lambda_p$ of Σ. These singular values can then be used to express the total variance as:

$$\text{Total Variance} = \operatorname{Tr}\Sigma = \operatorname{Tr} Q\Lambda Q^T = \operatorname{Tr}\Lambda = \lambda_1 + \cdots + \lambda_p.$$

The columns of Q are known as the *principal component directions* of our data and the singular values in Λ indicate how much variance each principal component explains. The first principal component (associated with the largest singular value) captures the most variance, the second principal component (associated with the second largest singular value) captures the second most variance, and so on. These principal components induce a new coordinate system for our data, in other words, each data point can now be represented using its *principal component score*, which are related to the original variables by the relations:

$$X = Q\,(PC) + \bar{x}$$

and

$$PC = Q^T(X - \bar{x}) \tag{6.1}$$

Here X is our original coordinate vector, PC is the coordinate vector corresponding to the principal component scores, and $\bar{x}$ is the vector of means of our dataset.

To illustrate these ideas, we will consider again the dataset shown in the following figure:

```
sample_data_plot
```

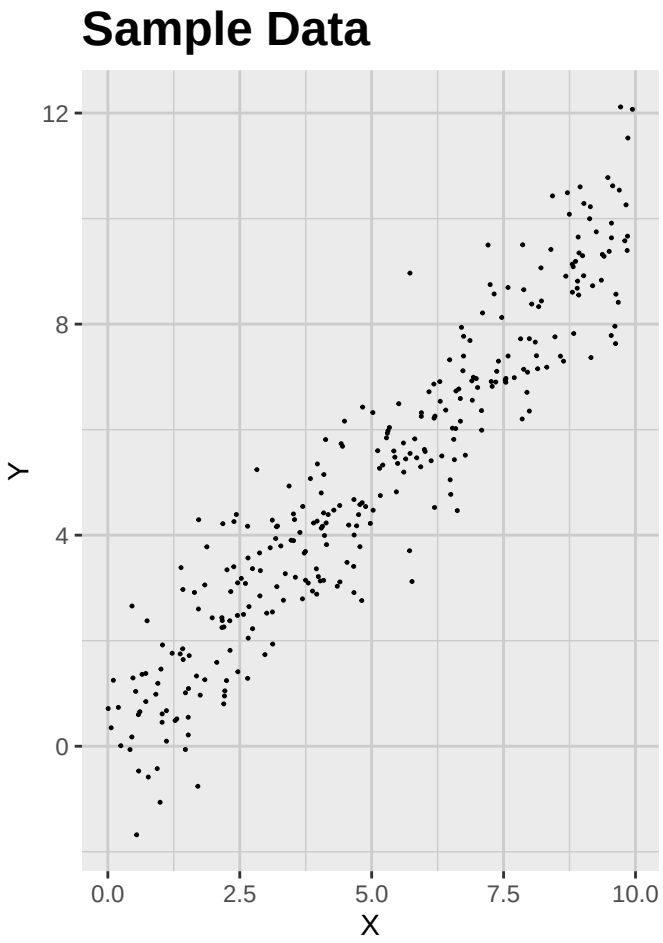

Figure 6.1: Sample data with two highly correlated variables.

Recall that this dataset consists of 300 observations of two variables, X and Y, the first one generated from a uniform distribution between 0 and 10, and the second one generated as $Y = X + \epsilon$, where ϵ is a normally distributed random error with mean 0 and standard deviation 1. The covariance matrix of this dataset is:

```
Sigma <- cov(sample_data_tbl)
```

$$\Sigma = \begin{bmatrix} 7.874751 & 7.7745959 \\ 7.7745959 & 8.6510962 \end{bmatrix}$$

The total variance is:

$$\text{Total Variance} = \operatorname{Tr}\Sigma = \Sigma_{11} + \Sigma_{22} = 16.5258472$$

We now look at the singular value decomposition of Σ, to obtain:

```
# Compute singular vectors and values
eig <- eigen(Sigma)
vectors <- eig$vectors
values <- eig$values
```

$$Q = \begin{bmatrix} 0.689251 & -0.7245227 \\ 0.7245227 & 0.689251 \end{bmatrix}$$

and

$$\Lambda = \begin{bmatrix} 16.0472039 & 0 \\ 0 & 0.4786433 \end{bmatrix}$$

The principal component directions are given by the columns of Q, and the amount of variance explained by each component is given by the corresponding singular value in Λ. In particular, since the vector of means is given by:

```
sample_mean_vector <- colMeans(sample_data_tbl)
```

$$\bar{x} = 4.9961185$$
$$\bar{y} = 5.019733$$

we have that the relation between the original variables and the principal component coordinates is given by:

$$X = 0.689251PC_1 + -0.7245227PC_2 + 4.9961185$$
$$Y = 0.7245227PC_1 + 0.689251PC_2 + 5.019733$$

and

$$PC_1 = 0.689251(X - 4.9961185) + 0.7245227(Y - 5.019733)$$
$$PC_2 = -0.7245227(X - 4.9961185) + 0.689251(Y - 5.019733)$$

Using these formulas, we can find the projection of our original data into the first and second principal components. The results are shown in the following figure:

```
sample_data_matrix <- as.matrix(sample_data_tbl)

# Center the data
centered_sample_data <- scale(sample_data_matrix, center = TRUE, scale = FALSE)

# SVD
```

```r
svd_result <- svd(centered_sample_data)
v1 <- svd_result$v[, 1]
v2 <- svd_result$v[, 2]

# Projection onto PC, in centered coordinates
proj_PC1 <- (centered_sample_data %*% v1) %*% t(v1)
proj_PC2 <- (centered_sample_data %*% v2) %*% t(v2)

# Shift back to original location
proj_coords1 <- proj_PC1 + attr(centered_sample_data, "scaled:center")
proj_coords2 <- proj_PC2 + attr(centered_sample_data, "scaled:center")

# Provide unique names before converting to tibble
colnames(proj_coords1) <- c("x_proj1", "y_proj1")
colnames(proj_coords2) <- c("x_proj2", "y_proj2")

proj_coords1_tbl <- as_tibble(proj_coords1)
proj_coords2_tbl <- as_tibble(proj_coords2)

# Bind with original data
compare_proj1_tbl <- bind_cols(sample_data_tbl, proj_coords1_tbl)
compare_proj2_tbl <- bind_cols(sample_data_tbl, proj_coords2_tbl)

# Generate plot corresponding to the first projection
compare_plot1 <- ggplot(compare_proj1_tbl) +
  geom_point(
    aes(x = x, y = y),
    color = "black",
    size = 0.2,
    alpha = 1
  ) +
  geom_point(
    aes(x = x_proj1, y = y_proj1),
    color = c_pal("C red"),
    size = 0.2,
    alpha = 1
  ) +
  labs(
    title = "First Principal Component Projection",
    x = "X",
    y = "Y"
  ) +
  coord_fixed() +
  theme_krul()
```

```r
# Generate plot corresponding to the second projection
compare_plot2 <- ggplot(compare_proj2_tbl) +
  geom_point(
    aes(x = x, y = y),
    color = "black",
    size = 0.2,
    alpha = 1
  ) +
  geom_point(
    aes(x = x_proj2, y = y_proj2),
    color = c_pal("C red"),
    size = 0.2,
    alpha = 1
  ) +
  labs(
    title = "Second Principal Component Projection",
    x = "X",
    y = "Y"
  ) +
  coord_fixed() +
  theme_krul()

# Combine plots
compare_plot1 + compare_plot2 +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Principal Component Projections",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

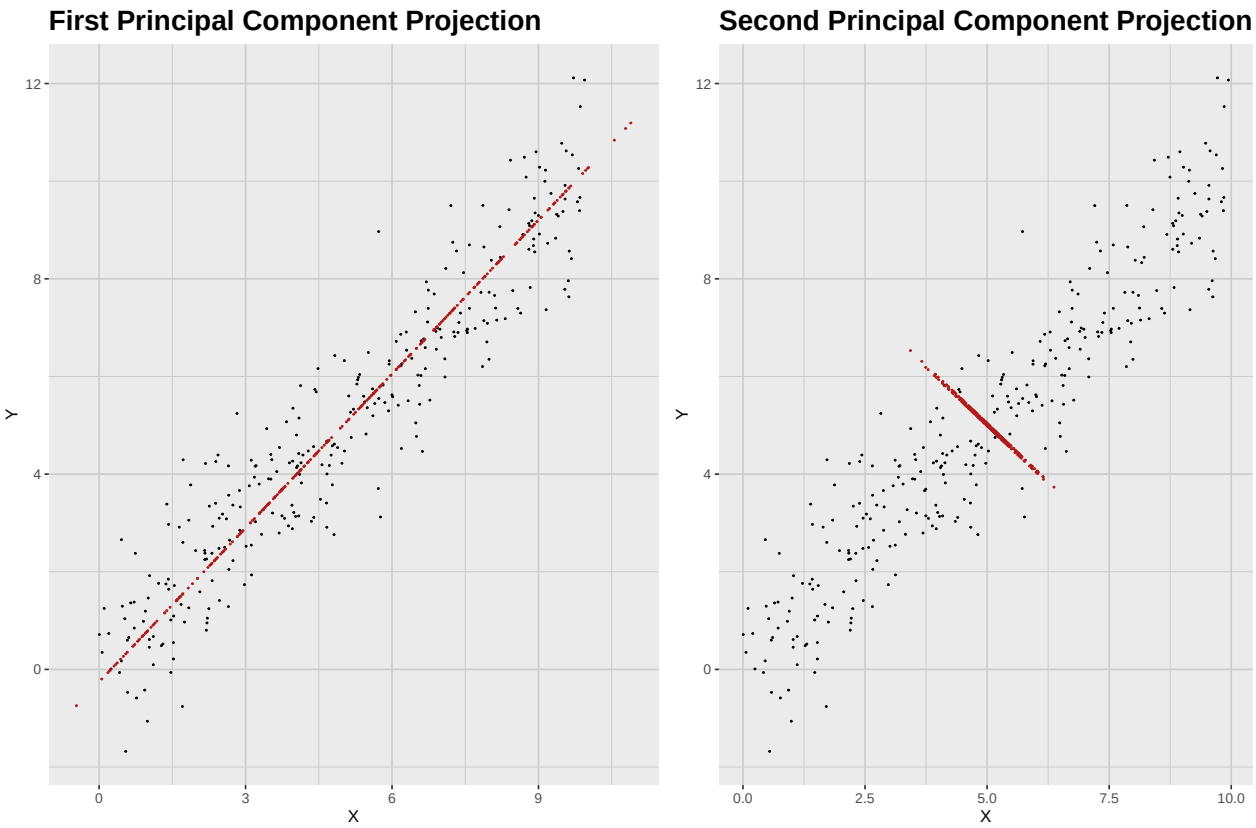

Figure 6.2: Projections onto the Principal Component Directions. Notice that much more information is lost when projecting onto the second principal component.

6.2 Cumulative Variance and Scree Plots

As we mentioned in the introduction to this chapter, PCA is often used for dimensionality reduction. This means that we want to reduce the number of variables in our dataset while retaining as much information as possible. To decide how many principal components to keep, we can look at the amount of variance explained by each component. A common way to visualize this information is through the use of a *scree plot*. The scree plot associated to a given dataset displays the singular values on the y-axis and the component number on the x-axis. It helps to identify the point where the explained variance starts to level off, often referred to as the "elbow". Another useful plot is the *cumulative variance plot*, which shows the cumulative proportion of variance explained by the principal components. Notice that the proportion of variance explained by the k-th principal component is given by:

$$\frac{\lambda_k}{\sum_{i=1}^{p} \lambda_i}$$

This plot helps to determine how many components are needed to explain a desired amount of total variance, such as 70% or 90%.

To illustrate this, we will use the well-known "iris" dataset, which contains measurements of four

features (sepal length, sepal width, petal length, and petal width) for three species of iris flowers (setosa, versicolor, and virginica). We will perform PCA on the four features and create a scree plot to visualize the variance explained by each principal component. First, we load the data and compute the covariance matrix for each species:

```
# Load the iris dataset
data(iris)

iris_setosa_tbl <- as_tibble(iris) %>%
  filter(Species == "setosa") %>%
  select(-Species)

iris_versicolor_tbl <- as_tibble(iris) %>%
  filter(Species == "versicolor") %>%
  select(-Species)

iris_virginica_tbl <- as_tibble(iris) %>%
  filter(Species == "virginica") %>%
  select(-Species)

# Compute covariance matrix
cov_matrix_setosa <- cov(iris_setosa_tbl)
cov_matrix_versicolor <- cov(iris_versicolor_tbl)
cov_matrix_virginica <- cov(iris_virginica_tbl)

# Singular value decomposition
eig_setosa <- eigen(cov_matrix_setosa)
eig_versicolor <- eigen(cov_matrix_versicolor)
eig_virginica <- eigen(cov_matrix_virginica)
```

The covariance matrices for each species are given by:

$$\Sigma_{setosa} = \begin{bmatrix} 0.12 & 0.1 & 0.02 & 0.01 \\ 0.1 & 0.14 & 0.01 & 0.01 \\ 0.02 & 0.01 & 0.03 & 0.01 \\ 0.01 & 0.01 & 0.01 & 0.01 \end{bmatrix}$$

$$\Sigma_{versicolor} = \begin{bmatrix} 0.27 & 0.09 & 0.18 & 0.06 \\ 0.09 & 0.1 & 0.08 & 0.04 \\ 0.18 & 0.08 & 0.22 & 0.07 \\ 0.06 & 0.04 & 0.07 & 0.04 \end{bmatrix}$$

$$\Sigma_{virginica} = \begin{bmatrix} 0.4 & 0.09 & 0.3 & 0.05 \\ 0.09 & 0.1 & 0.07 & 0.05 \\ 0.3 & 0.07 & 0.3 & 0.05 \\ 0.05 & 0.05 & 0.05 & 0.08 \end{bmatrix}$$

The singular values for each species are given by:

$$\Lambda_{setosa} = \begin{bmatrix} 0.24 & 0 & 0 & 0 \\ 0 & 0.04 & 0 & 0 \\ 0 & 0 & 0.03 & 0 \\ 0 & 0 & 0 & 0.01 \end{bmatrix}$$

$$\Lambda_{versicolor} = \begin{bmatrix} 0.49 & 0 & 0 & 0 \\ 0 & 0.07 & 0 & 0 \\ 0 & 0 & 0.05 & 0 \\ 0 & 0 & 0 & 0.01 \end{bmatrix}$$

$$\Lambda_{virginica} = \begin{bmatrix} 0.7 & 0 & 0 & 0 \\ 0 & 0.11 & 0 & 0 \\ 0 & 0 & 0.05 & 0 \\ 0 & 0 & 0 & 0.03 \end{bmatrix}$$

The associated principal component directions are given by the columns of the matrices

$$Q_{setosa} = \begin{bmatrix} -0.67 & 0.6 & 0.44 & -0.04 \\ -0.73 & -0.62 & -0.27 & -0.02 \\ -0.1 & 0.49 & -0.83 & -0.24 \\ -0.06 & 0.13 & -0.2 & 0.97 \end{bmatrix}$$

$$Q_{versicolor} = \begin{bmatrix} -0.69 & 0.67 & -0.27 & 0.1 \\ -0.31 & -0.57 & -0.73 & -0.23 \\ -0.62 & -0.34 & 0.63 & -0.32 \\ -0.21 & -0.34 & 0.06 & 0.92 \end{bmatrix}$$

$$Q_{virginica} = \begin{bmatrix} -0.74 & -0.17 & -0.53 & 0.37 \\ -0.2 & 0.75 & -0.33 & -0.54 \\ -0.63 & -0.17 & 0.65 & -0.39 \\ -0.12 & 0.62 & 0.43 & 0.65 \end{bmatrix}$$

The scree plot and cumulative variance plot for the three species are shown in the following figure:

```
# Scree plot data
scree_data <- tibble(
  Component = 1:4,
  Setosa = eig_setosa$values,
  Versicolor = eig_versicolor$values,
  Virginica = eig_virginica$values
) %>%
  pivot_longer(
    cols = -Component,
    names_to = "Species",
    values_to = "SingularValue"
  )
# Scree plot
scree_plot <- scree_data %>%
```

```
  ggplot(aes(x = Component, y = SingularValue, color = Species)) +
  geom_point(size = 3) +
  geom_line() +
  c_scale_color("C red", "C blue", "C green") +
  scale_x_continuous(breaks = 1:4) +
  labs(
    title = "Scree Plot of Iris Dataset",
    x = "Principal Component",
    y = "Singular Value"
  ) +
  theme_krul() +
  theme(legend.position = "top")

# Cumulative variance data
cumulative_variance_data <- scree_data %>%
  group_by(Species) %>%
  arrange(Component) %>%
  mutate(CumulativeVariance = cumsum(SingularValue) / sum(SingularValue))
# Cumulative variance plot
cumulative_variance_plot <- cumulative_variance_data %>%
  ggplot(aes(x = Component, y = CumulativeVariance, color = Species)) +
  geom_point(size = 3) +
  geom_line() +
  c_scale_color("C red", "C blue", "C green") +
  scale_x_continuous(breaks = 1:4) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(
    title = "Cumulative Variance Explained by Principal Components",
    x = "Principal Component",
    y = "Cumulative Variance Explained"
  ) +
  theme_krul() +
  theme(legend.position = "top")

# Combine scree plot and cumulative variance plot
scree_plot + cumulative_variance_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Scree Plot and Cumulative Variance Plot for the Iris Dataset",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
```

```
)
```

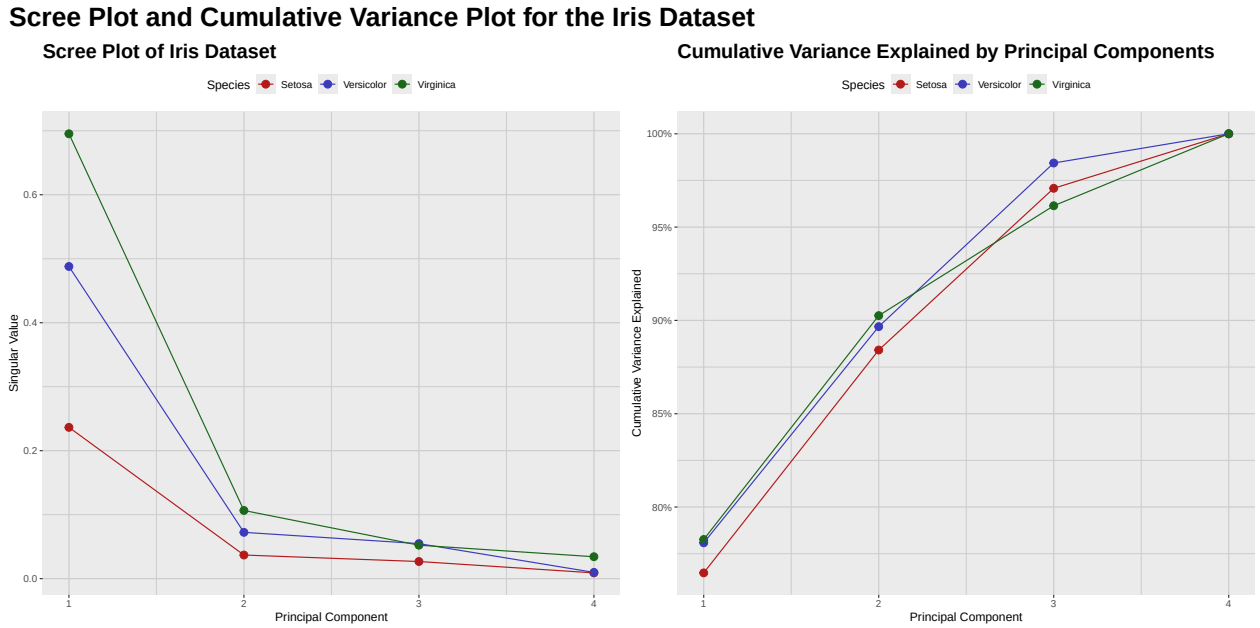

Figure 6.3: Principal Component Analysis of the Iris Dataset by Species. To decide the number of components to keep, we can use the scree plot (left) and look for the 'elbow' where the explained variance levels off. Alternatively, we can use the cumulative variance plot (right) to determine how many components are needed to explain a desired amount of total variance, such as 70% or 90%.

6.3 Loadings and Biplots

Recall that equation (6.1) gives the relation between the original variables and the principal component scores. Using this equation, we can quickly compute these scores for each observation in our dataset. The coefficients appearing in this equation are known as the *loadings* of the principal components, and they indicate how much each original variable contributes to each principal component. A*biplot* is a graphical representation that displays both the principal component scores of the observations and the loadings of the variables on the same plot. This allows us to visualize how the original variables contribute to the principal components and how the observations are distributed in the space defined by these components.

To illustrate this concept, we will create biplots for the first two principal components of the "iris" dataset for each species. We start by finding the loadings corresponding to each species, for which

we simply take the first two columns of the matrices Q_{setosa}, $Q_{versicolor}$, and $Q_{virginica}$ shown above.

$$\text{Loadings}_{setosa} = \begin{bmatrix} -0.67 & 0.6 \\ -0.73 & -0.62 \\ -0.1 & 0.49 \\ -0.06 & 0.13 \end{bmatrix}$$

$$\text{Loadings}_{versicolor} = \begin{bmatrix} -0.69 & 0.67 \\ -0.31 & -0.57 \\ -0.62 & -0.34 \\ -0.21 & -0.34 \end{bmatrix}$$

$$\text{Loadings}_{virginica} = \begin{bmatrix} -0.74 & -0.17 \\ -0.2 & 0.75 \\ -0.63 & -0.17 \\ -0.12 & 0.62 \end{bmatrix}$$

We can then compute the principal component scores for each observation by simply multiplying the values of our original variables by these matrices. The resulting biplots for each species are shown in the following figure:

```
# Function to prepare biplot data from precomputed singular vectors
prepare_biplot_from_eigen <- function(tbl, eig) {
  data_matrix <- as.matrix(tbl)
  centered_data_matrix <- scale(data_matrix, center = TRUE, scale = FALSE)


  # Scores: project observations onto PC1 and PC2
  scores <- centered_data_matrix %*% eig$vectors[, 1:2]
  colnames(scores) <- c("PC1", "PC2")
  scores_tbl <- as_tibble(scores) %>%
    mutate(Obs = row_number())


  # Loadings: first two singular vectors
  loadings <- eig$vectors[, 1:2]
  colnames(loadings) <- c("PC1", "PC2")
  loadings <- as_tibble(loadings) %>%
    mutate(Variable = colnames(tbl))

  list(scores = scores_tbl, loadings = loadings)
}

# Prepare data for each species
setosa_data <- prepare_biplot_from_eigen(
  iris_setosa_tbl,
  eig_setosa
)
```

```
versicolor_data <- prepare_biplot_from_eigen(
  iris_versicolor_tbl,
  eig_versicolor
)

virginica_data <- prepare_biplot_from_eigen(
  iris_virginica_tbl,
  eig_virginica
)

# Function to create biplot
create_biplot <- function(scores_tbl, loadings_tbl, species_name) {
  ggplot() +
    geom_point(
      data = scores_tbl,
      aes(x = PC1, y = PC2),
      color = c_pal("C red"),
      alpha = 1,
      size = 0.3
    ) +
    geom_segment(
      data = loadings_tbl,
      aes(
        x = 0,
        y = 0,
        xend = PC1 * max(scores_tbl$PC1),
        yend = PC2 * max(scores_tbl$PC2)
      ),
      arrow = arrow(length = unit(0.3, "cm")),
      color = c_pal("C blue"),
    ) +
    geom_text(
      data = loadings_tbl,
      aes(
        x = PC1 * max(scores_tbl$PC1) * 1.1,
        y = PC2 * max(scores_tbl$PC2) * 1.1,
        label = Variable
      ),
      color = "black",
      size = 5
    ) +
    labs(
      title = paste("Biplot for", species_name),
      x = "PC1",
```

```
      y = "PC2"
    ) +
    theme_minimal()
}

# Create biplots
biplot_setosa <- create_biplot(
  setosa_data$scores,
  setosa_data$loadings,
  "Setosa"
)

biplot_versicolor <- create_biplot(
  versicolor_data$scores,
  versicolor_data$loadings,
  "Versicolor"
)

biplot_virginica <- create_biplot(
  virginica_data$scores,
  virginica_data$loadings,
  "Virginica"
)

# Combine biplots
biplot_setosa + biplot_versicolor + biplot_virginica +
  plot_layout(ncol = 3) +
  plot_annotation(
    title = "Biplots of Iris Species (PC1 vs PC2)",
    theme = theme(
      plot.title = element_text(size = 24, face = "bold")
    )
  )
```

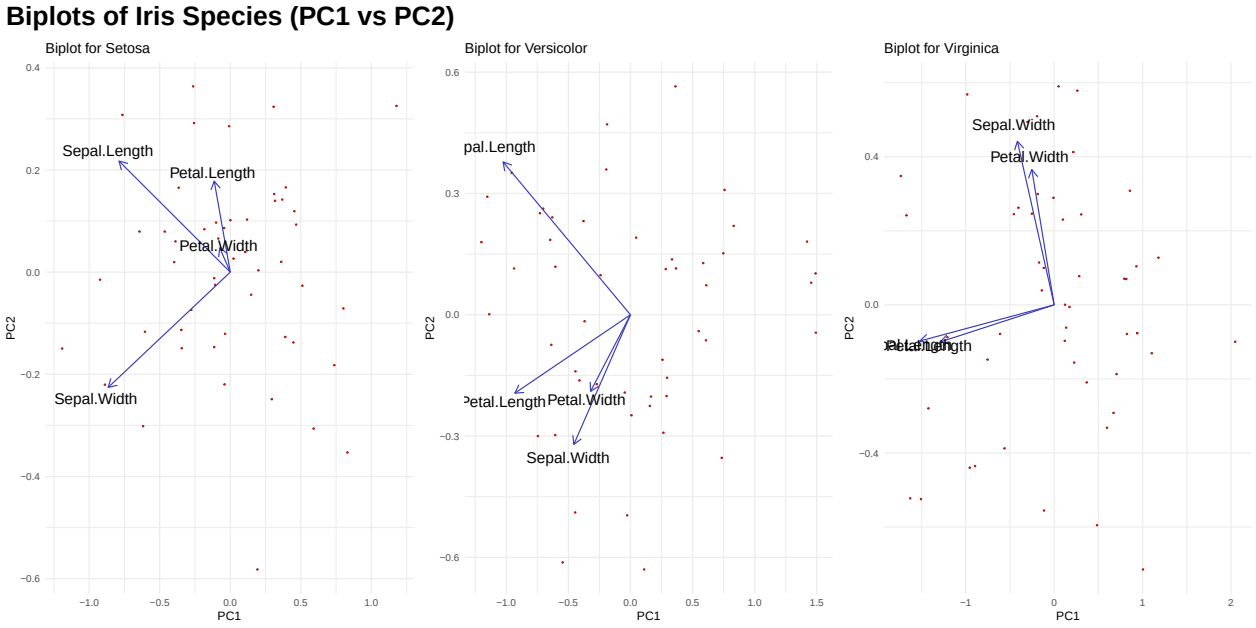

Figure 6.4: Loadings and scores for the first two principal components of the Iris dataset by species.

6.4 Summary of the PCA Process

In general, when performing PCA, it is recommended to follow these steps:

1. **Standardize the data (if needed).** It is not always easy to decide whether to standardize the data or not. Here are some guidelines:
   - If the variables are measured on different scales (e.g., height in cm and weight in kg), standardization is usually recommended.
   - If the variables are on the same scale and have similar variances, standardization may not be necessary.
   - If the variables have very different variances, standardization can help to give equal weight to all variables.
2. **Compute the covariance matrix.**
   - The matrix captures how variables vary together.
   - Its size is $p \times p$, where p is the number of variables.
3. **Find the singular value decomposition (SVD) of the covariance matrix.**
   - Eigenvalues measure the variance explained by each principal component.
   - Eigenvectors give the directions (linear combinations of variables) of the components.
4. **Rank the components.**
   - Order the singular values from largest to smallest.

- The singular vector corresponding to the largest singular value is the first principal component.

5. **Decide how many components to keep.**
   - *Kaiser criterion*: keep the singular values > 1 (for correlation matrix).
   - *Scree plot*: look for the "elbow" where explained variance levels off.
   - *Cumulative variance*: keep enough components to explain about $70\% - 90\%$ of the total variance.
6. **Find the principal component scores and loadings.**
   - Project the original data onto the selected principal components.
   - Use the formula: $PC = Q^T(X - \bar{x})$.
7. **Interpret the components.**
   - Examine the *loadings* to see which variables contribute the most.
   - Larger absolute values indicate stronger influence.
   - Use biplots to visualize scores and loadings together.
   - Apply dimension reduction before regression, clustering, or other analyses.
   - Apply noise reduction by discarding low-variance components.

Part III

Factor Analysis

Chapter 7

Factor Analysis: Theoretical Models

Factor Analysis is a statistical technique that is widely used in applied fields to identify hidden patterns or latent structures behind large sets of observed variables. The goal is to reduce complexity by grouping related variables into a smaller number of underlying *factors*, which are easier to interpret.

Some areas where factor analysis is commonly applied include:

- **Psychology and social sciences:** to uncover latent traits such as intelligence, anxiety, personality dimensions, or satisfaction levels, based on responses to questionnaires or surveys.
- **Marketing and consumer research:** to identify purchasing motives, brand perceptions, or market segments from customer behavior data.
- **Finance and economics:** to reveal hidden drivers of stock returns, macroeconomic indicators, or risk factors.
- **Education:** to analyze test items and determine whether they measure common skills or abilities.
- **Biological and health sciences:** to find latent structures in genetic data, symptom checklists, or patient-reported outcomes.

In practice, Factor Analysis is used when researchers suspect that a few unobserved dimensions are responsible for the correlations among many observed variables. It is especially useful for building measurement scales, data reduction, and gaining insights into the structure of complex datasets.

7.1 Factor Analysis Setting

The theoretical context for factor analysis is a relation of the form

$$X = \mu + LF + \epsilon$$

where:

- X is a random vector of p observed variables (indicators).
- μ is a constant vector of dimension p.

- L is a $p \times m$ constant matrix known as the matrix of *factor loadings.*
- $F \sim \mathcal{N}(0, I_m)$ is a vector of m latent factors (unobserved variables) that are assumed to be normally distributed with mean zero and identity covariance matrix.
- $\epsilon \sim \mathcal{N}(0, \Psi)$ is a vector of unique errors (specific variances) that are also assumed to be normally distributed with mean zero and diagonal covariance matrix $\Psi = \text{diag}(\sigma_1^2, \ldots, \sigma_p^2)$.
- The factors F and the errors ϵ are assumed to be independent, in particular $\text{Cov}[F, \epsilon] = 0$.

As a consequence of these assumptions, we have that

$$\mathbb{E}[X] = \mathbb{E}[\mu + LF + \epsilon] = \mathbb{E}[\mu] + L\mathbb{E}[F] + \mathbb{E}[\epsilon] = \mu$$

and

$$\begin{aligned}
\text{Var}[X] &= \text{Var}[LF + \epsilon] \\
&= \mathbb{E}[(LF + \epsilon)(LF + \epsilon)^T] \\
&= \mathbb{E}[LFF^TL^T] + \mathbb{E}[\epsilon\epsilon^T] + \mathbb{E}[LF\epsilon^T] + \mathbb{E}[\epsilon F^TL^T] \\
&= L\mathbb{E}[FF^T]L^T + \mathbb{E}[\epsilon\epsilon^T] + L\mathbb{E}[F\epsilon^T] + \mathbb{E}[\epsilon F^T]L^T \\
&= LI_mL^T + \Psi + L0 + 0L^T \\
&= LL^T + \Psi
\end{aligned}$$

Furthermore, since X is a linear combination of normally distributed variables, it follows that

$$X \sim \mathcal{N}(\mu, LL^T + \Psi)$$

7.2 Variance Decomposition

The variance decomposition $\text{Var}[X] = LL^T + \Psi$ shows that the total variance of the observed variables X can be decomposed into two components:

- **Common variance (LL^T):** This part of the variance is explained by the common factors F. It captures the shared variability among the observed variables that can be attributed to the underlying latent structure.
- **Unique variance (Ψ):** This part of the variance is specific to each observed variable and is not explained by the common factors. It includes measurement errors and any other variability that is unique to each variable.

In particular, the variance of each individual observed variable X_i can be expressed as

$$\text{Var}[X_i] = \sum_{j=1}^{m} L_{ij}^2 + \sigma_i^2$$

where $h_i^2 = \sum_{j=1}^{m} L_{ij}^2$ is called the *communality* of the variable X_i, representing the portion of its variance explained by the common factors, and σ_i^2 is the unique variance specific to X_i.

7.3 Factor Score Estimation

Assume that we have a model

$$X = \mu + LF + \epsilon$$

with the same conditions as before. Then the joint distribution of X and F is given by

$$\begin{bmatrix} X \\ F \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu \\ 0 \end{bmatrix}, \begin{bmatrix} LL^T + \Psi & L \\ L^T & I_m \end{bmatrix} \right)$$

Notice that, in particular,

$$\text{Cov}[X, F] = \mathbb{E}[(X - \mu)(F - 0)^T] = \mathbb{E}[(LF + \epsilon)F^T] = L\mathbb{E}[FF^T] + \mathbb{E}[\epsilon F^T] = LI_m + 0 = L$$

Using the properties of the multivariate normal distribution, we can derive the conditional distribution of the factors F given the observed variables X. This conditional distribution is also normal, with mean and covariance given by

$$[F|X = x] \sim \mathcal{N}\left(L^T(LL^T + \Psi)^{-1}(x - \mu), I_m - L^T(LL^T + \Psi)^{-1}L\right)$$

The mean of this conditional distribution,

$$\mathbb{E}[F|X = x] = L^T(LL^T + \Psi)^{-1}(x - \mu)$$

is known as the *factor score* for the observed data point $X = x$. It provides an estimate of the latent factors based on the observed variables and the factor loadings. The covariance matrix

$$I_m - L^T(LL^T + \Psi)^{-1}L$$

represents the uncertainty associated with the factor score estimates.

Remark 7.3.1. Notice that, if

$$f = \mathbb{E}[F|X = x] = L^T(LL^T + \Psi)^{-1}(x - \mu)$$

then, in general, the *residual*

$$r(x) = x - \mu - Lf$$

is different from zero. In fact, as a function of the random variable X, the residual has probability distribution

$$r(X) \sim \mathcal{N}\left(0, L\left(I_m + L^T\Psi^{-1}L\right)^{-1}L^T + \Psi\right)$$

7.4 Rotation of Factors

An important aspect of factor analysis is that the factors are not uniquely defined. If F is a vector of factors, then any orthogonal transformation $F^* = QF$, where Q is an orthogonal matrix (i.e., $Q^TQ = I$), will yield a new set of factors that explain the same variance in the observed variables. In particular,

$$\text{Var}[F^*] = \text{Var}[QF] = Q\,\text{Var}[F]Q^T = QI_mQ^T = I_m$$

The corresponding factor loadings will be transformed as $L^* = LQ^T$. In other words, if

$$X = \mu + LF + \epsilon$$

is a valid factor model, then so is

$$X = \mu + L^*F^* + \epsilon = \mu + LQ^TQF + \epsilon = \mu + LF + \epsilon$$

This means that the solution to a factor analysis problem is not unique, and different rotations of the factors can lead to different interpretations. In the next chapter, we will explore various practical methods for factor analysis and rotation techniques geared towards achieving more interpretable factor solutions.

Chapter 8

Factor Analysis: Practical Methods

Recall from last chapter that the classical factor analysis model is given by

$$X = \mu + LF + \epsilon$$

where:

- X is a random vector of p observed variables (indicators).
- μ is a constant vector of dimension p.
- L is a $p \times m$ constant matrix known as the matrix of *factor loadings.*
- $F \sim \mathcal{N}(0, I_m)$ is a vector of m latent factors (unobserved variables) that are assumed to be normally distributed with mean zero and identity covariance matrix.
- $\epsilon \sim \mathcal{N}(0, \Psi)$ is a vector of unique errors (specific variances) that are also assumed to be normally distributed with mean zero and diagonal covariance matrix $\Psi = \text{diag}(\sigma_1^2, \ldots, \sigma_p^2)$.
- The factors F and the errors ϵ are assumed to be independent, in particular $\text{Cov}[F, \epsilon] = 0$.

Notice that, since $\epsilon \sim \mathcal{N}(0, \Psi)$, this model is completely specified by the parameters μ, L, and Ψ. In this chapter we will discuss practical methods for estimating these parameters from data, as well as methods for estimating the latent factors F (known as *factor scores*) for our observations once the model parameters have been estimated.

8.1 Assessing Factorability

The first step in applying factor analysis to a dataset is to assess whether the data is suitable for factor analysis, i.e., whether the observed variables exhibit sufficient correlations to justify the use of a factor model. Several statistical tests and measures can be used to evaluate the factorability of a dataset:

1. **Correlation Matrix Examination:** A preliminary step is to visually inspect the correlation matrix of the observed variables. Look for patterns of high correlations (e.g., > 0.3) among variables, which suggest that they may share common underlying factors.

2. **Bartlett's Test of Sphericity:** This test evaluates the hypothesis that the correlation matrix is an identity matrix, which would indicate that the variables are uncorrelated and unsuitable for factor analysis. A significant result (p-value < 0.05) suggests that the variables are correlated enough to proceed with factor analysis.

3. **Kaiser-Meyer-Olkin (KMO) Measure of Sampling Adequacy:** The KMO statistic measures the proportion of variance in the observed variables that can be attributed to underlying common factors, relative to variance that is unique to each variable. It ranges from 0 to 1, with values closer to 1 indicating that factor analysis is likely to be useful. A KMO value above 0.6 is generally considered acceptable for factor analysis.

8.2 Deciding the Number of Factors

Determining the appropriate number of factors to retain in a factor analysis is a crucial step that can significantly impact the results and interpretations. Several methods can be employed to decide on the number of factors:

1. **Eigenvalue Criterion (Kaiser Criterion):** Retain factors with singular values greater than 1. This method is based on the idea that a factor should explain at least as much variance as a single observed variable.

2. **Scree Plot:** A scree plot displays the singular values in descending order. Look for an "elbow" point where the slope of the curve changes dramatically. The number of factors to retain is typically determined by the number of points before this elbow.

3. **Parallel Analysis:** This method involves comparing the observed singular values with those obtained from random data of the same size. Factors are retained if their singular values exceed those from the random data.

4. **Model Fit Indices:** For more advanced factor analysis methods (e.g., confirmatory factor analysis), various fit indices (e.g., RMSEA, TLI) can be used to assess how well the model fits the data for different numbers of factors.

8.3 Extracting Factors

Once the number of factors has been determined, the next step is to extract the factors from the data. Several methods are available for factor extraction:

1. **Principal Axis Factoring (PAF):** This method focuses on the shared variance among variables and is suitable when the goal is to identify underlying constructs. It iteratively estimates communalities and extracts factors based on the reduced correlation matrix.

2. **Maximum Likelihood (ML) Factor Analysis:** This method estimates factor loadings by maximizing the likelihood of the observed data given the model. It allows for statistical testing of hypotheses about the factor structure and provides goodness-of-fit indices.

8.4 Rotation of Factors

After extracting factors, it is often beneficial to rotate them to achieve a simpler and more interpretable structure. Rotation can be either orthogonal (factors remain uncorrelated) or oblique (factors are allowed to correlate). Common rotation methods include:

1. **Varimax Rotation:** An orthogonal rotation method that maximizes the variance of squared loadings within each factor, leading to a simpler structure where each variable loads highly on one factor and low on others.
2. **Quartimax Rotation:** Another orthogonal rotation method that simplifies the rows of the loading matrix, aiming to have each variable load highly on as few factors as possible.
3. **Equamax Rotation:** A hybrid orthogonal rotation method that combines the goals of varimax and quartimax, balancing the simplification of both rows and columns of the loading matrix.
4. **Promax Rotation:** An oblique rotation method that allows factors to correlate. It starts with a varimax rotation and then raises the loadings to a power to achieve a simpler structure.

8.5 Computing Factor Scores

Once the factor loadings have been estimated and the factors have been rotated (if applicable), the next step is to compute factor scores for each observation in the dataset. Factor scores represent the estimated values of the latent factors for each individual based on their observed variable scores. Several methods can be used to compute factor scores:

1. **Regression Method:** This method estimates factor scores by regressing the observed variables on the estimated factor loadings. It provides unbiased estimates of factor scores but may not preserve the original variance of the factors.
2. **Bartlett Method:** This method computes factor scores by minimizing the residual variance of the observed variables. It tends to produce factor scores that are more reliable and have lower measurement error.
3. **Anderson-Rubin Method:** This method generates factor scores that are uncorrelated and standardized. It is particularly useful when the factors are assumed to be orthogonal.

8.6 Assessing Model Fit

After fitting a factor analysis model, it is important to assess how well the model fits the data. Several statistical tests and fit indices can be used to evaluate model fit:

1. **Chi-Square Test of Model Fit:** This test evaluates the null hypothesis that the model-implied covariance matrix matches the observed covariance matrix. A non-significant result (p-value > 0.05) suggests that the model fits the data well.
2. **Root Mean Square Error of Approximation (RMSEA):** This index measures the discrepancy between the observed and model-implied covariance matrices per degree of freedom. Values less than 0.06 indicate a good fit.

3. **Tucker-Lewis Index (TLI):** This index compares the fit of the specified model to a null model. Values greater than 0.95 indicate a good fit.
4. **Standardized Root Mean Square Residual (SRMR):** This index measures the average discrepancy between observed and predicted correlations. Values less than 0.08 indicate a good fit.

8.7 Visualizing the Model

Visualizing the results of a factor analysis can help in interpreting the factors and understanding the relationships among variables. Common visualization techniques include:

1. **Scree Plot:** A scree plot displays the singular values in descending order, helping to visualize the number of factors to retain.
2. **Factor Loading Plot:** A scatter plot of factor loadings can help visualize how variables load on different factors, especially after rotation.
3. **Biplot:** A biplot combines factor scores and loadings in a single plot, allowing for the visualization of both observations and variable relationships in the factor space.
4. **Heatmap of Loadings:** A heatmap can be used to visualize the magnitude of factor loadings across variables and factors, highlighting patterns and clusters.

Remark 8.7.1. Notice that Factor Loading Plots and Biplots only make sense when we have 2 or 3 factors, otherwise we cannot visualize them properly.

8.8 Example: Factor Analysis of the Holzinger-Swineford Dataset

We will now provide an example of factor analysis using the "HolzingerSwineford1939" dataset from the "lavaan" package. This dataset contains scores from a battery of 9 cognitive tests administered to 301 students in grades 7 and 8. The tests were designed to measure 3 latent abilities: visual/spatial, verbal, and speed. For this example we will focus only on tests x4 to x9 so that we can fit a two-factor model and visualize the results in a biplot. The exact description of each test is given below:

1. **Paragraph (x4)**: Reading comprehension. Students read short paragraphs and answer questions about the main idea, details, and simple inferences.
2. **Sentence (x5)**: Sentence completion. Choose the word or phrase that best completes a sentence so it is clear and grammatical; emphasizes vocabulary-in-context and syntax.
3. **Word Meaning (x6)**: Vocabulary. Select the best synonym/definition for a target word; reflects breadth of lexical knowledge.
4. **Addition (x7)**: Timed arithmetic fluency. Complete many simple addition problems quickly and accurately; captures processing speed with minimal reasoning load.
5. **Counting Dots (x8)**: Rapid visual enumeration. Quickly count small groups of dots across many items; measures visual scanning and speeded accuracy.

6. **Straight–Curved Capitals (x9)**: Perceptual discrimination speed. Scan capital letters and mark those with only straight strokes (versus any curved strokes), under strict time limits.

To perform this analysis, we will start by loading the necessary libraries and the dataset:

```
# Load required libraries
library(lavaan)
library(psych)
library(scales)

# Load the Holzinger-Swineford1939 dataset
data(HolzingerSwineford1939)

# Select the relevant tests
hs_data <- as_tibble(HolzingerSwineford1939) %>%
  select("x4", "x5", "x6", "x7", "x8", "x9")
```

8.8.1 Assessing Factorability

We start our factor analysis process by assessing the factorability for tests x4–x9. First we collect all the required information:

```
# Correlation matrix (pairwise complete observations)
Sigma <- cor(hs_data)

# Bartlett's test and KMO measure
n_eff <- nrow(hs_data)
bart <- cortest.bartlett(Sigma, n = n_eff)
kmo  <- KMO(Sigma)
```

We now show the correlation heatmap:

```
corr_df <- Sigma %>%
  as_tibble(rownames = "Var1") %>%
  pivot_longer(-Var1, names_to = "Var2", values_to = "r")

# Preserve matrix order on axes
corr_df$Var1 <- factor(corr_df$Var1, levels = colnames(Sigma))
corr_df$Var2 <- factor(corr_df$Var2, levels = colnames(Sigma))

ggplot(corr_df, aes(x = Var2, y = Var1, fill = r)) +
  geom_tile(color = "black") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 3) +
  scale_fill_gradient2(
    low = c_pal("C blue"), mid = "white", high = c_pal("C red"),
    midpoint = 0, limits = c(-1, 1)
  ) +
  coord_fixed() +
```

```
  labs(
    title = "Correlation heatmap for tests x4-x9",
    x = "", y = "", fill = "Correlation"
  ) +
  theme(
    plot.title = element_text(size = 18, face = "bold"),
    panel.background = element_blank(),
    panel.grid = element_blank(),
    axis.ticks = element_blank()
  )
```

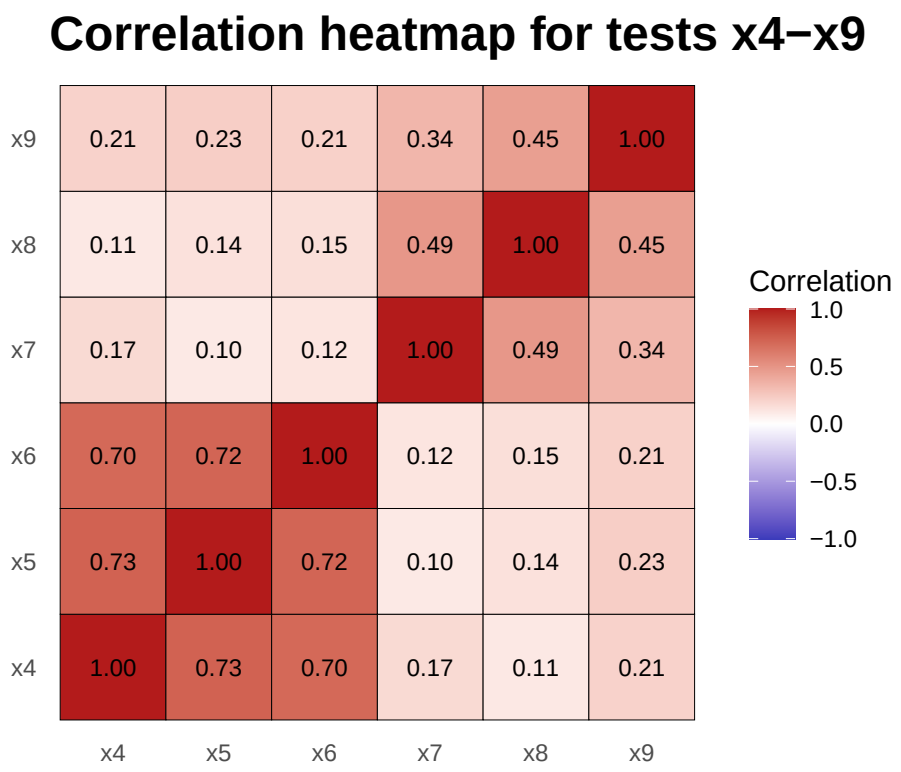

Figure 8.1: Correlation heatmap for tests x4–x9. Notice the strong correlations among the verbal tests (x4, x5, x6) and the speed tests (x7, x8, x9), suggesting that a two-factor model may be appropriate for this data.

Notice that there is a strong correlation among the verbal tests (x4, x5, x6) and among the speed tests (x7, x8, x9), but not such strong correlation between the two groups. This is a good indication that a two-factor model may be appropriate for this data. We now report the results of Bartlett's test and the KMO measure:

$$\text{Bartlett's test of sphericity} = 0$$
$$\text{Kaiser-Meyer-Olkin measure} = 0.7308774$$

As we can see, all tests so far indicate that factor analysis is appropriate for this dataset.

8.8.2 Deciding the Number of Factors

We will now use a scree plot and a cumulative variance plot to help us decide on the number of factors to retain. Since we are using the correlation matrix, we will look for singular values greater than 1 (Kaiser criterion) and also look for an "elbow" in the scree plot.

```
# Scree analysis from the correlation matrix
eig <- eigen(Sigma)

scree_df <- tibble(
  component = seq_along(eig$values),
  singular_value = eig$values
) %>%
  mutate(
    cumulative = cumsum(singular_value) / sum(singular_value)
  )

# Scree plot
scree_plot <- scree_df %>%
  ggplot(aes(x = component, y = singular_value)) +
  geom_point(size = 3, color = c_pal("C red")) +
  geom_line(color = c_pal("C blue")) +
  geom_hline(
    yintercept = 1,
    linetype = "dashed",
    color = c_pal("C green"),
    linewidth = 1.5
  ) +
  scale_x_continuous(breaks = scree_df$component) +
  labs(
    title = "Scree Plot",
    x = "Principal Component",
    y = "Singular Value"
  ) +
  theme_krul()

# Cumulative variance plot
cum_plot <- ggplot(scree_df, aes(x = component, y = cumulative)) +
  geom_point(size = 3, color = c_pal("C red")) +
  geom_line(color = c_pal("C blue")) +
  scale_x_continuous(breaks = scree_df$component) +
  scale_y_continuous(
    labels = percent_format(accuracy = 1),
    limits = c(0, 1)
  ) +
  labs(
```

```
    title = "Cumulative Variance Plot",
    x = "Principal Component",
    y = "Cumulative Variance Explained"
  ) +
  theme_krul()

# Combine scree plot and cumulative variance plot
scree_plot + cum_plot + plot_layout(ncol = 2) +
  plot_annotation(
    title = paste(
      "Scree Plot and Cumulative Variance Plot for the",
      "Holzinger-Swineford Dataset"
    ),
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

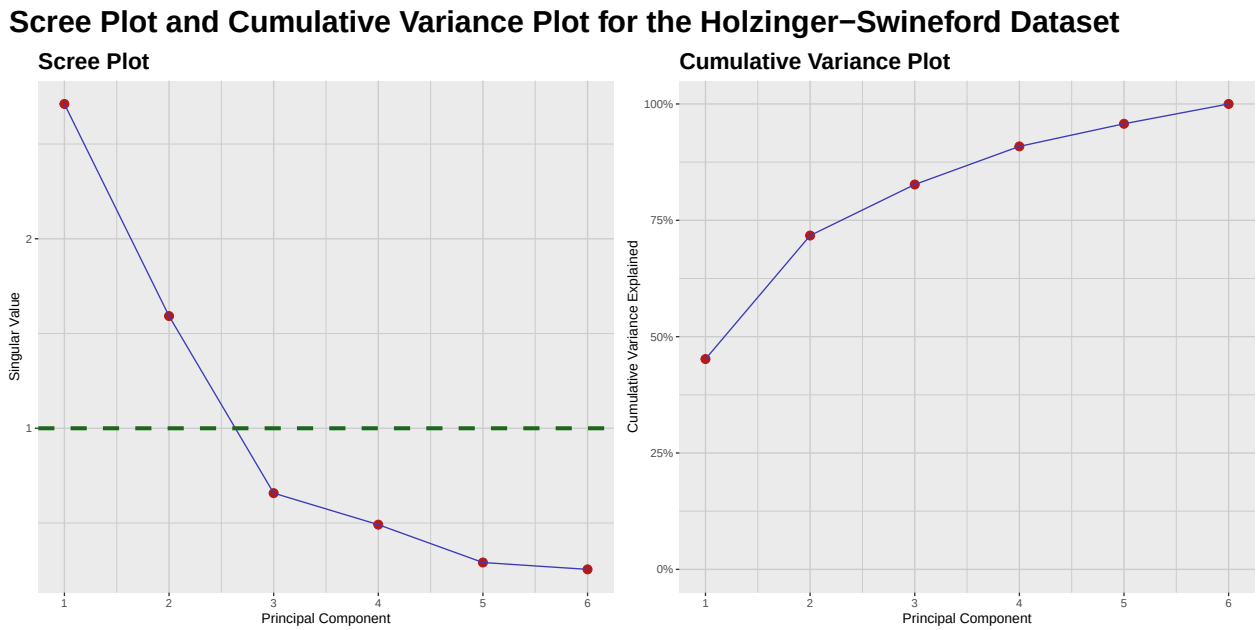

Figure 8.2: Scree plot and cumulative variance for tests x4–x9. Both plots suggest that a two-factor solution is appropriate for this dataset.

Notice that both the scree plot and the cumulative variance plot suggest that a two-factor solution is appropriate for analysing the six tests included in this dataset.

8.8.3 Extracting, Rotating, and Scoring Factors

Even though extracting, rotating and scoring factors are theoretically separate steps, in practice they are often performed together using specialized software or statistical packages. Here we will use the `psych::fa` function to perform all three steps at once. We will fit a 2-factor model using maximum likelihood (ML) estimation, compute factor scores using the regression method, and perform a varimax rotation to simplify the factor structure.

```
# Fit 2-factor FA using ML, regression scores and varimax rotation
fa_rot <- fa(
  r = hs_data,
  nfactors = 2,
  rotate = "varimax",
  fm = "ml",
  scores = "regression",
  use = "pairwise"
)

# Extract factor loadings into a tibble
loadings <- fa_rot$loadings[, 1:2] %>%
  unclass() %>%
  as_tibble(rownames = "Variable") %>%
  rename(F1 = 2, F2 = 3)
# Extract factor scores into a tibble
scores <- fa_rot$scores %>%
  as_tibble() %>%
  setNames(c("F1", "F2")) %>%
  mutate(Obs = row_number())
```

8.8.4 Assessing Model Fit

We now evaluate how well the 2-factor ML solution reproduces the observed correlations. For this, we perform the tests χ-squared, RMSEA, CFI, TLI, and RMSR.

```
# Extract standard fit indices from psych::fa object (ML)
chisq <- fa_rot$STATISTIC
df    <- fa_rot$dof
pval  <- fa_rot$PVAL
rmsea <- as.numeric(fa_rot$RMSEA[1])
tli   <- fa_rot$TLI
bic   <- fa_rot$BIC

L   <- as.matrix(fa_rot$loadings)
Phi <- if (!is.null(fa_rot$Phi)) fa_rot$Phi else diag(ncol(L))
uni <- if (!is.null(fa_rot$uniquenesses)) {
  fa_rot$uniquenesses
} else {
```

```
  1 - rowSums((L %*% Phi) * L)
}
Rhat <- L %*% Phi %*% t(L) + diag(as.numeric(uni))

# Helper function to extract lower-triangle entries (off-diagonal)
lt_offdiag <- function(M) M[lower.tri(M, diag = FALSE)]

# Classic RMSR
res <- Sigma - Rhat
rmsr <- sqrt(mean(lt_offdiag(res)^2, na.rm = TRUE))

# Standardized RMSR (SRMR)
is_cov <- any(abs(diag(Sigma) - 1) > 1e-8)
S_obs_cor  <- if (is_cov) stats::cov2cor(Sigma) else Sigma
S_hat_cor  <- if (is_cov) stats::cov2cor(Rhat)  else Rhat

res_cor <- S_obs_cor - S_hat_cor

# SRMR = sqrt( sum_{i<j}(res_ij^2) / sum_{i<j}(obs_ij^2) )
num <- sum(lt_offdiag(res_cor)^2, na.rm = TRUE)
den <- sum(lt_offdiag(S_obs_cor)^2, na.rm = TRUE)
srmr <- sqrt(num / den)
```

The results of these tests are:

$$\begin{aligned} \chi^2 \text{ Test} &= 0.1156716 \\ \text{RMSEA} &= 0.0531269 \\ \text{TLI} &= 0.9804573 \\ \text{RMSR} &= 0.0411927 \end{aligned}$$

All tests indicate that the 2-factor model fits the data well.

8.8.5 Visualizing the Model

Finally, we will visualize the model using a biplot. We will also rotate the factors manually to illustrate what could happen if we did not choose an appropriate rotation method.

```
# Scaling factor for arrows (keep consistent across plots)
arrow_scale <- 3

# Base (unrotated) biplot
p_base <- ggplot() +
  geom_point(
    data = scores,
    aes(x = F1, y = F2),
    alpha = 1,
```

```
    color = c_pal("C red"),
    size = 0.5
  ) +
  geom_segment(
    data = loadings,
    aes(x = 0, y = 0, xend = F1 * arrow_scale, yend = F2 * arrow_scale),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C blue"),
    linewidth = 1.2
  ) +
  geom_text(
    data = loadings,
    aes(x = F1 * arrow_scale, y = F2 * arrow_scale, label = Variable),
    color = "black",
    hjust = -0.5
  ) +
  labs(
    title = "Varimax Rotation",
    x = "Factor 1",
    y = "Factor 2"
  ) +
  theme_krul()

# 45° rotation of scores and loadings
theta  <- pi / 4
cos_t  <- cos(theta)
sin_t  <- sin(theta)

scores_rot <- scores %>%
  mutate(
    F1_rot = F1 * cos_t - F2 * sin_t,
    F2_rot = F1 * sin_t + F2 * cos_t
  )

loadings_rot <- loadings %>%
  mutate(
    F1_rot = F1 * cos_t - F2 * sin_t,
    F2_rot = F1 * sin_t + F2 * cos_t
  )

# Rotated biplot
p_rot <- ggplot() +
  geom_point(
    data = scores_rot,
    aes(x = F1_rot, y = F2_rot),
```

```
    alpha = 1,
    color = c_pal("C red"),
    size = 0.5
  ) +
  geom_segment(
    data = loadings_rot,
    aes(x = 0, y = 0, xend = F1_rot * arrow_scale, yend = F2_rot * arrow_scale),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C blue"),
    linewidth = 1.2
  ) +
  geom_text(
    data = loadings_rot,
    aes(x = F1_rot * arrow_scale, y = F2_rot * arrow_scale, label = Variable),
    color = "black",
    hjust = -0.5
  ) +
  labs(
    title = "Manual Rotation",
    x = "Rotated Factor 1",
    y = "Rotated Factor 2"
  ) +
  theme_krul()

# Optional: share axis limits for fair comparison
xr <- range(
  c(
    scores$F1,
    scores_rot$F1_rot,
    loadings$F1 * arrow_scale,
    loadings_rot$F1_rot * arrow_scale
  ),
  na.rm = TRUE
)

yr <- range(
  c(
    scores$F2,
    scores_rot$F2_rot,
    loadings$F2 * arrow_scale,
    loadings_rot$F2_rot * arrow_scale
  ),
  na.rm = TRUE
)
p_base <- p_base + coord_equal(xlim = xr, ylim = yr)
```

```
p_rot  <- p_rot  + coord_equal(xlim = xr, ylim = yr)

# Side-by-side using patchwork
p_base + p_rot + plot_layout(ncol = 2) +
  plot_annotation(
    title = paste(
      "Biplots of the factor analysis model",
      "using Varimax and Manual Rotations"
    ),
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

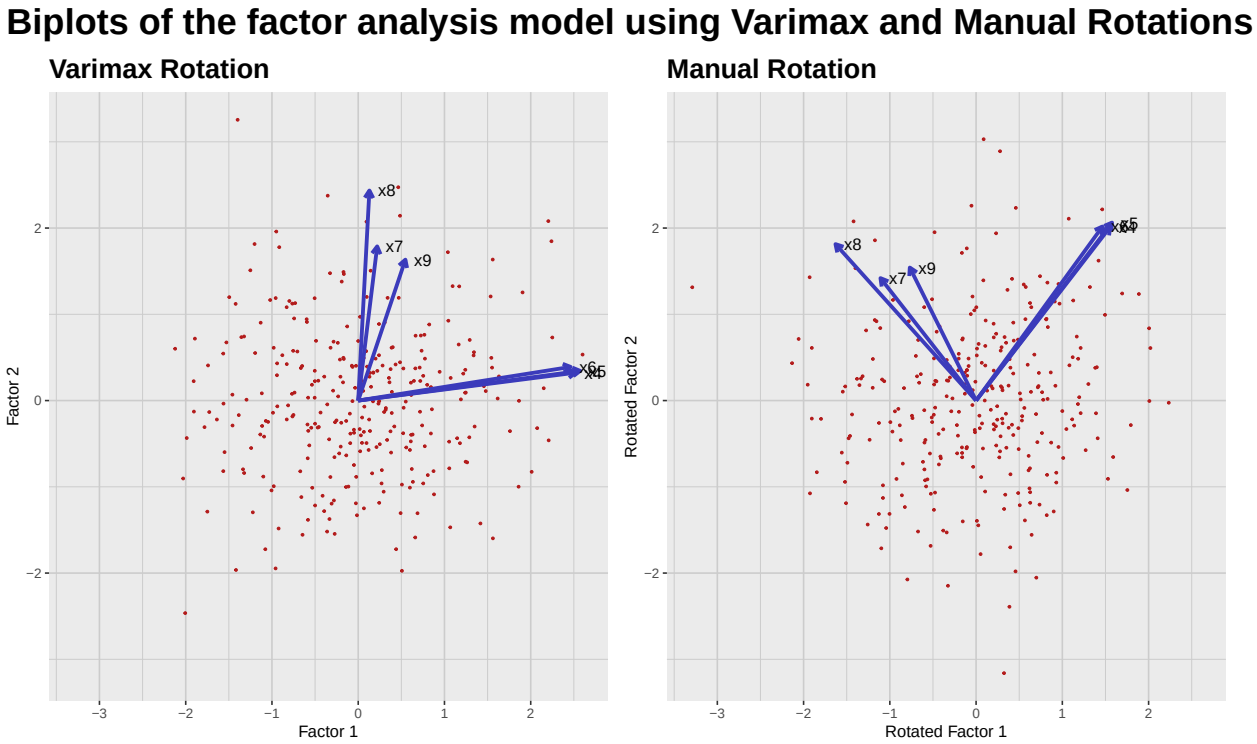

Figure 8.3: Biplots of the 2-factor solution for tests x4–x9. Left: Varimax rotation. Right: Manual 45° rotation. Notice how the varimax rotation leads to a simpler and more interpretable structure, with verbal tests (x4, x5, x6) loading highly on Factor 1 and speed tests (x7, x8, x9) loading highly on Factor 2.

Part IV

Cluster Analysis

Chapter 9

Hierarchical Clustering

Clustering is an unsupervised learning technique that groups similar data points together based on their features. Unlike supervised learning, clustering does not rely on predefined labels or categories. Instead, it identifies patterns and structures within the data itself. Clustering methods are usually categorized into two main families:

1. **Hierarchical Clustering**: These methods create a *dendrogram* (also known as a *tree diagram*) representing the taxonomic structure of our data. It is specially useful for small datasets and when the number of clusters is unknown.
2. **Non-Hierarchical Clustering**: These methods, such as K-means and DBSCAN, partition the data into a predefined number of clusters or based on density. They are generally more scalable for larger datasets.

In this chapter, we will focus on hierarchical clustering, leaving the non-hierarchical methods for the next one.

9.1 Understanding Dendrograms

A dendrogram is a tree-like diagram that illustrates the taxonomic structure of our data according to a hierarchical clustering algorithm. It provides a visual representation of how data points are grouped together based on their similarities. The vertical axis represents the distance or dissimilarity between clusters, while the horizontal axis represents the individual data points or clusters. By cutting the dendrogram at a specific height, we can determine the number of clusters that we want to consider for our analysis.

As an illustration, lets consider the scatter plot of the Iris dataset (considering only the variables "Sepal.Length" and "Sepal.Width") where each point is colored according to the species of the flower:

```
data(iris)

iris_plot <- iris %>%
  ggplot(aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_point(
```

```
    size = 1.3,
    alpha = 1
  ) +
  c_scale_color("C green", "C blue", "C red") +
  labs(
    title = "Iris Dataset: Sepal Length vs Sepal Width",
    x = "Sepal Length (cm)",
    y = "Sepal Width (cm)"
  ) +
  theme_krul() +
  theme(legend.position = "top")

iris_plot
```

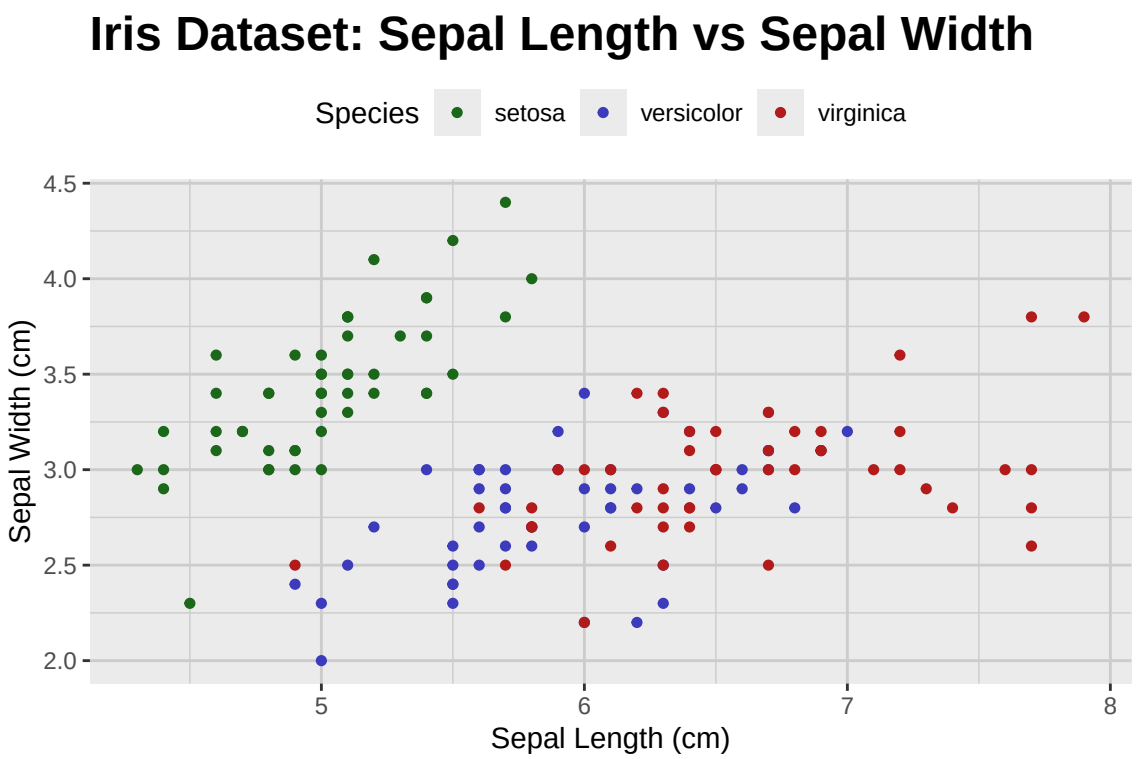

Figure 9.1: Scatter plot of the Iris dataset, separated by species and considering only the variables 'Sepal.Length' and 'Sepal.Width'.

We will now consider a dedrogram associated to this dataset and show how cutting it at different heights leads to different numbers of clusters:

```
# Use the built-in iris dataset only
iris_tbl <- as_tibble(iris)

k_values <- c(1, 3, 4, 6)
```

```r
# Cluster on the chosen 2D projection to match the plot
standardized_sepal_matrix <- iris_tbl %>%
  select(Sepal.Length, Sepal.Width) %>%
  scale()

distances <- stats::dist(standardized_sepal_matrix, method = "euclidean")
sepal_hclust <- stats::hclust(distances, method = "ward.D2")

sepal_tbl <- iris_tbl %>%
  select(Sepal.Length, Sepal.Width) %>%
  mutate(.row_id = row_number())

# Build long data for facets by k
long_pts <- lapply(k_values, function(k) {
  cluster <- stats::cutree(sepal_hclust, k = k)
  tibble::tibble(
    .row_id = seq_along(cluster),
    cluster = factor(as.integer(cluster)),
    k = factor(paste0("k = ", k), levels = paste0("k = ", k_values))
  )
})

long_pts <- bind_rows(long_pts)
sepal_cluster_tbl <- left_join(long_pts, sepal_tbl, by = ".row_id")

# Polygons for each (k, cluster)
sepal_poly_tbl <- cluster_polygons(
  sepal_cluster_tbl,
  Sepal.Length, Sepal.Width,
  k, cluster
)

sepal_cluster_plot <- sepal_cluster_tbl %>%
  ggplot(aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(color = "black", alpha = 1, size = 0.8) +
  labs(
    title = "Clusters",
    x = "Sepal Length",
    y = "Sepal Width"
  ) +
  theme_krul() +
  facet_wrap(~k, ncol = 1, scales = "fixed")

sepal_cluster_plot <- sepal_cluster_plot +
```

```
  geom_path(
    data = sepal_poly_tbl,
    aes(
      x = Sepal.Length,
      y = Sepal.Width,
      group = interaction(k, cluster),
      color = cluster
    ),
    linewidth = 0.8,
    alpha = 0.5
  ) +
  c_scale_color(
    "C green",
    "C blue",
    "C red",
    "C magenta",
    "C yellow",
    "C cyan"
  ) +
  labs(color = "Cluster")

# Dendrogram geometry (replicated across facets)
sepal_dendro_data <- ggdendro::dendro_data(sepal_hclust, type = "rectangle")
k_fac <- factor(paste0("k = ", k_values), levels = paste0("k = ", k_values))

dendro_segments_faceted <- sepal_dendro_data$segments %>%
  mutate(.rep = 1) %>%
  tidyr::crossing(k = k_fac) %>%
  select(-.rep)

# One dashed cut line per k facet
dendro_cuts_tbl <- tibble::tibble(
  k = k_fac,
  y_cut = vapply(
    k_values,
    function(k) cut_dendrogram_height_for_k(sepal_hclust, k),
    numeric(1)
  )
)

dendro_faceted <- ggplot() +
  geom_segment(
    data = dendro_segments_faceted,
    aes(x = x, y = y, xend = xend, yend = yend),
    linewidth = 0.4
```

```
  ) +
  geom_hline(
    data = dendro_cuts_tbl,
    aes(yintercept = y_cut),
    linetype = "dashed",
    color = c_pal("C red")
  ) +
  labs(title = "Dendrogram cuts", x = NULL, y = "Height") +
  theme_krul() +
  theme(
    axis.text.x = element_blank(),
    axis.ticks.x = element_blank(),
    panel.grid = element_blank()
  ) +
  facet_wrap(~k, ncol = 1, scales = "fixed")

combined_plot <-  (dendro_faceted | sepal_cluster_plot) +
  plot_layout(widths = c(1, 2)) +
  plot_annotation(
    title = "Hierarchical Clustering of the Iris Dataset",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

combined_plot
```

Hierarchical Clustering of the Iris Dataset

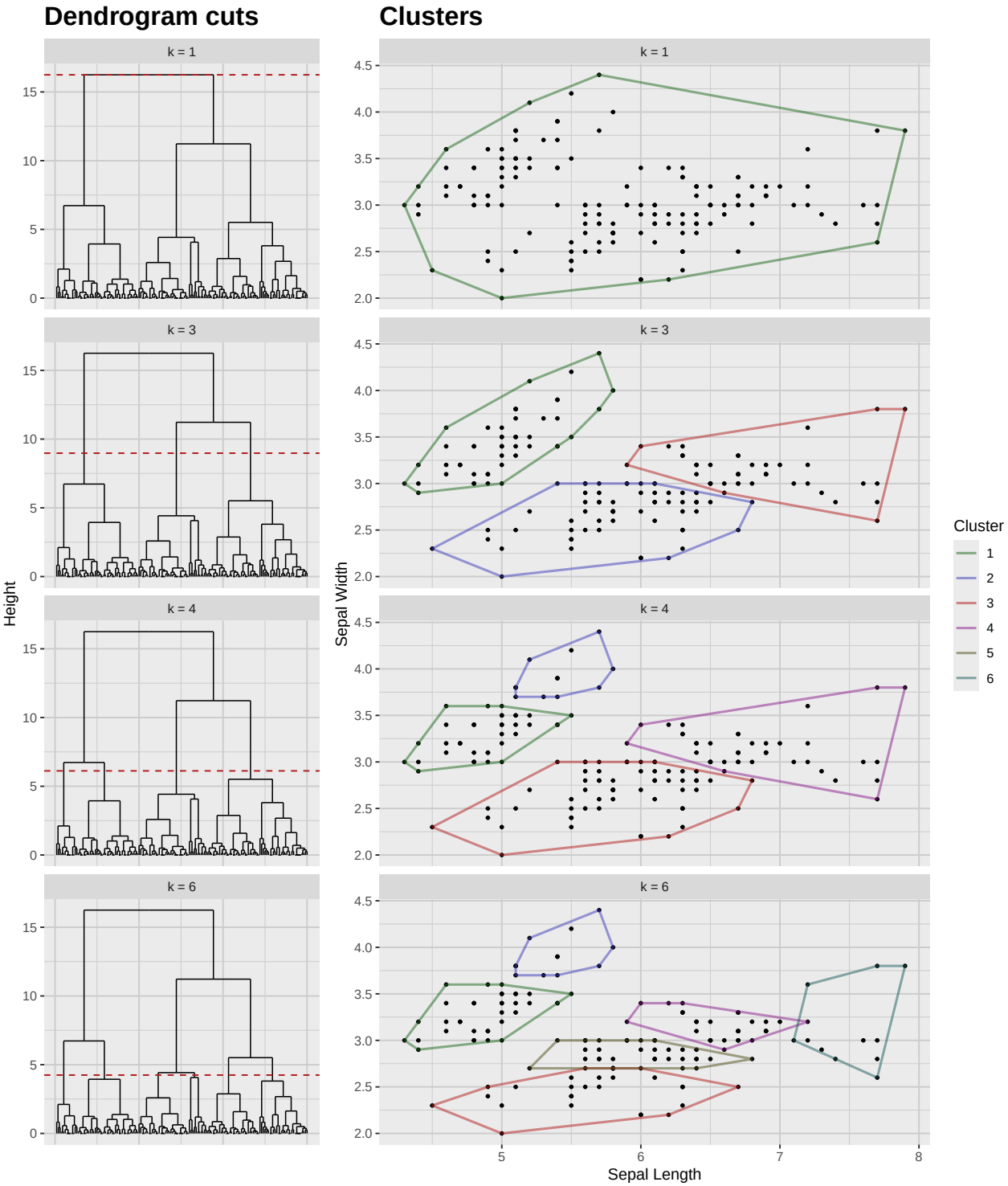

Figure 9.2: Hierarchical clustering of the Iris dataset considering only the variables 'Sepal.Length' and 'Sepal.Width'. The dashed lines indicate the cut points for the given numbers of clusters (k = 1, 3, 4, 6).

9.2 Performing Hierarchical Clustering

There are several methods to perform hierarchical clustering, but the most common ones are:

- **Agglomerative Clustering**: This is a bottom-up approach where each data point starts as its own cluster, and pairs of clusters are merged iteratively based on their similarity until a single cluster is formed or a stopping criterion is met.
- **Divisive Clustering**: This is a top-down approach where all data points start in a single cluster, and the cluster is recursively split into smaller clusters based on dissimilarity until each data point is its own cluster or a stopping criterion is met.

Once we have chosen the method, we need to define a distance metric to measure the similarity between data points. Common distance metrics include:

- **Euclidean Distance**: This is the most common distance metric, which calculates the straight-line distance between two points in a multi-dimensional space.
- **Mahalanobis Distance**: This metric takes into account the correlations between variables and is useful when dealing with multivariate data.
- **Canberra Distance**: This metric is sensitive to small changes in the data and is useful when dealing with sparse data. The formula for the Canberra distance between two points p and q is given by:

  $$d(p,q) = \sum_{i=1}^{n} \frac{|p_i - q_i|}{|p_i| + |q_i|}$$

  where n is the number of dimensions, and p_i and q_i are the coordinates of points p and q respectively.
- **Manhattan Distance**: Also known as the L^1 norm or taxicab distance, this metric calculates the distance between two points by summing the absolute differences of their coordinates. It is particularly useful when dealing with high-dimensional data or when the data has a grid-like structure. The formula for the Manhattan distance between two points p and q is given by:

  $$d(p,q) = \sum_{i=1}^{n} |p_i - q_i|$$

- **Minkowski Distance**: This is a generalization of the Euclidean and Manhattan distances, which allows for different values of the parameter p to control the distance calculation. The formula for the Minkowski distance between two points p and q is given by:

  $$d(p,q) = \left(\sum_{i=1}^{n} |p_i - q_i|^p \right)^{1/p}$$

  When $p = 1$, it corresponds to the Manhattan distance, and when $p = 2$, it corresponds to the Euclidean distance.

Finally, we need to choose a linkage criterion to determine how the distance between clusters is calculated. Common linkage criteria include:

- **Single Linkage**: This criterion defines the distance between two clusters as the minimum distance between any pair of points from each cluster. It tends to create long, chain-like clusters and is sensitive to noise and outliers.
- **Complete Linkage**: This criterion defines the distance between two clusters as the maximum distance between any pair of points from each cluster. It tends to create compact, spherical clusters and is less sensitive to noise and outliers compared to single linkage.
- **Average Linkage**: This criterion defines the distance between two clusters as the average distance between all pairs of points from each cluster. It provides a balance between single and complete linkage and is less sensitive to noise and outliers.
- **Ward's Linkage**: This criterion minimizes the total within-cluster variance by merging clusters that result in the smallest increase in variance. It tends to create compact, spherical clusters and is less sensitive to noise and outliers compared to other linkage criteria.

Once we have chosen the method, distance metric, and linkage criterion, we can perform hierarchical clustering using the `hclust` function in R. As was shown in the previous example, we can then visualize the results using a dendrogram and cut the dendrogram at a specific height to determine the number of clusters.

Chapter 10

Non-Hierarchical Clustering

In *non-hierarchical clustering*, we partition the data into a predetermined number of clusters. In this case, we don't obtain a dendrogram, but rather a flat set of clusters. There are many methods for non-hierarchical clustering, but most of them can be classified as *partitioning methods*, *density-based methods* or *model-based methods*.

10.1 Partitioning Methods

These methods create a partition of the data into a set of k clusters, where k is specified by the user. They rely on optimizing some objective function that measures the quality of the clustering. The most common partitioning methods are:

- **k-means clustering**: This method aims to partition the data into k clusters such that the sum of squared distances between each point and the centroid of its assigned cluster is minimized. The algorithm iteratively assigns points to the nearest centroid and updates the centroids until convergence.
- **k-medoids clustering**: Similar to k-means, but instead of using the mean of the points in a cluster as the centroid, it uses an actual data point (the medoid) that minimizes the sum of distances to all other points in the cluster. This makes k-medoids more robust to noise and outliers.
- **CLARA (Clustering LARge Applications)**: An extension of k-medoids that is designed to handle large datasets by sampling a subset of the data and applying the k-medoids algorithm to this sample.
- **PAM (Partitioning Around Medoids)**: A specific implementation of the k-medoids algorithm that uses a greedy approach to find the medoids and assign points to clusters.

10.2 Density-Based Methods

These methods identify clusters based on the density of points in the data space. They can find clusters of arbitrary shapes and are robust to noise. The most common density-based methods are:

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**: This method defines clusters as areas of high density separated by areas of low density. It requires two parameters: the radius ϵ that defines the neighborhood around a point, and the minimum number of points required to form a dense region. Points that do not belong to any cluster are considered noise.
- **OPTICS (Ordering Points To Identify the Clustering Structure)**: An extension of DBSCAN that creates an ordering of the points based on their density, allowing for the identification of clusters at different density levels.
- **Mean Shift**: This method iteratively shifts each point towards the mean of the points in its neighborhood, effectively finding the modes of the data distribution. Clusters are formed around these modes.

10.3 Model-Based Methods

These methods assume that the data is generated from a mixture of underlying probability distributions, and they aim to identify these distributions and assign points to clusters based on their likelihood of belonging to each distribution. The most common model-based methods are:

- **Gaussian Mixture Models (GMM)**: This method models the data as a mixture of several Gaussian distributions, each representing a cluster. The parameters of the distributions are estimated using the Expectation-Maximization (EM) algorithm, and points are assigned to clusters based on their posterior probabilities.
- **Bayesian Clustering**: This method uses Bayesian inference to estimate the parameters of the underlying distributions and to assign points to clusters. It can incorporate prior knowledge about the data and can handle uncertainty in the clustering process.
- **Latent Class Analysis (LCA)**: This method is used for categorical data and assumes that the data is generated from a mixture of latent classes. It estimates the parameters of the classes and assigns points to classes based on their posterior probabilities.

10.4 Choosing the Right Method

The choice of non-hierarchical clustering method depends on the characteristics of the data and the specific goals of the analysis. Here are some guidelines for choosing the right method:

- If the data is continuous and the clusters are expected to be spherical and of similar size, k-means or k-medoids may be appropriate.
- If the data contains noise or outliers, or if the clusters are expected to be of different shapes and sizes, density-based methods like DBSCAN or OPTICS may be more suitable.
- If the data is believed to be generated from a mixture of underlying distributions, model-based methods like GMM or Bayesian clustering may be the best choice.
- If the data is categorical, Latent Class Analysis (LCA) may be the most appropriate method.

- If the dataset is large, consider using methods like CLARA that are designed to handle large datasets efficiently.

10.5 Evaluation of Clustering Results

Evaluating the quality of clustering results is crucial to ensure that the chosen method has effectively captured the underlying structure of the data. Here are some common techniques for evaluating clustering results:

- **Internal Validation**: This involves using metrics that assess the quality of the clustering based on the data itself, without reference to external information. Common internal validation metrics include:
  - *Silhouette Score*: Measures how similar a point is to its own cluster compared to other clusters. A higher silhouette score indicates better-defined clusters.
  - *Davies-Bouldin Index*: Evaluates the average similarity ratio of each cluster with its most similar cluster. Lower values indicate better clustering.
  - *Calinski-Harabasz Index*: Measures the ratio of between-cluster variance to within-cluster variance. Higher values indicate better-defined clusters.
- **External Validation**: This involves comparing the clustering results to known labels or ground truth, if available. Common external validation metrics include:
  - *Adjusted Rand Index (ARI)*: Measures the similarity between the clustering results and the ground truth, adjusted for chance. Higher values indicate better agreement.
  - *Normalized Mutual Information (NMI)*: Measures the amount of information shared between the clustering results and the ground truth. Higher values indicate better agreement.
- **Stability Analysis**: This involves assessing the robustness of the clustering results by applying the clustering algorithm to different subsets of the data or by adding noise to the data. Consistent clustering results across different runs indicate stable clusters.
- **Visual Inspection**: Visualizing the clustering results using scatter plots, heatmaps, or dendrograms can provide insights into the structure of the clusters and help identify any potential issues with the clustering.

10.6 Example in R

We will now demonstrate how the different non-hierarchical clustering methods can be implemented in R using the `iris` dataset. We will start by loading the necessary libraries.

```
library(cluster)
library(dbscan)
library(mclust)
library(krulRutils)
```

Next, we will prepare the data and show the plot comparing the results of different clustering methods.

```
set.seed(123)

# Use only Sepal.Width and Sepal.Length
iris_tbl <- as_tibble(iris) %>%
  select(Sepal.Width, Sepal.Length)

iris_matrix <- as.matrix(iris_tbl)

# k-means (partitioning)
iris_kmeans <- kmeans(iris_matrix, centers = 3, nstart = 50)$cluster

# PAM (k-medoids, partitioning)
iris_pam <- pam(iris_matrix, k = 3)$clustering

# GMM (model-based, Gaussian mixtures)
gmm <- Mclust(iris_matrix, G = 3)$classification


db_raw <- choose_dbscan(iris_matrix, min_pts = 5)
db <- assign_noise_to_nearest(iris_matrix, db_raw$cluster)

# Bind results for faceted plotting
labs <- list(
  `k-means` = iris_kmeans,
  PAM = iris_pam,
  GMM = gmm,
  DBSCAN = db
)

plot_df <- bind_rows(lapply(names(labs), function(m) {
  tibble(
    sepal_width = iris_tbl$Sepal.Width,
    sepal_length = iris_tbl$Sepal.Length,
    method = m,
    cluster = factor(labs[[m]])
  )
}))

ggplot(plot_df, aes(sepal_width, sepal_length, color = cluster)) +
  geom_point(size = 1.3, alpha = 1) +
  c_scale_color("C red", "C green", "C blue") +
  facet_wrap(~ method, ncol = 2) +
  labs(title = "Iris sepal-based clustering",
```

```
      x = "Sepal Width", y = "Sepal Length") +
theme_krul()
```

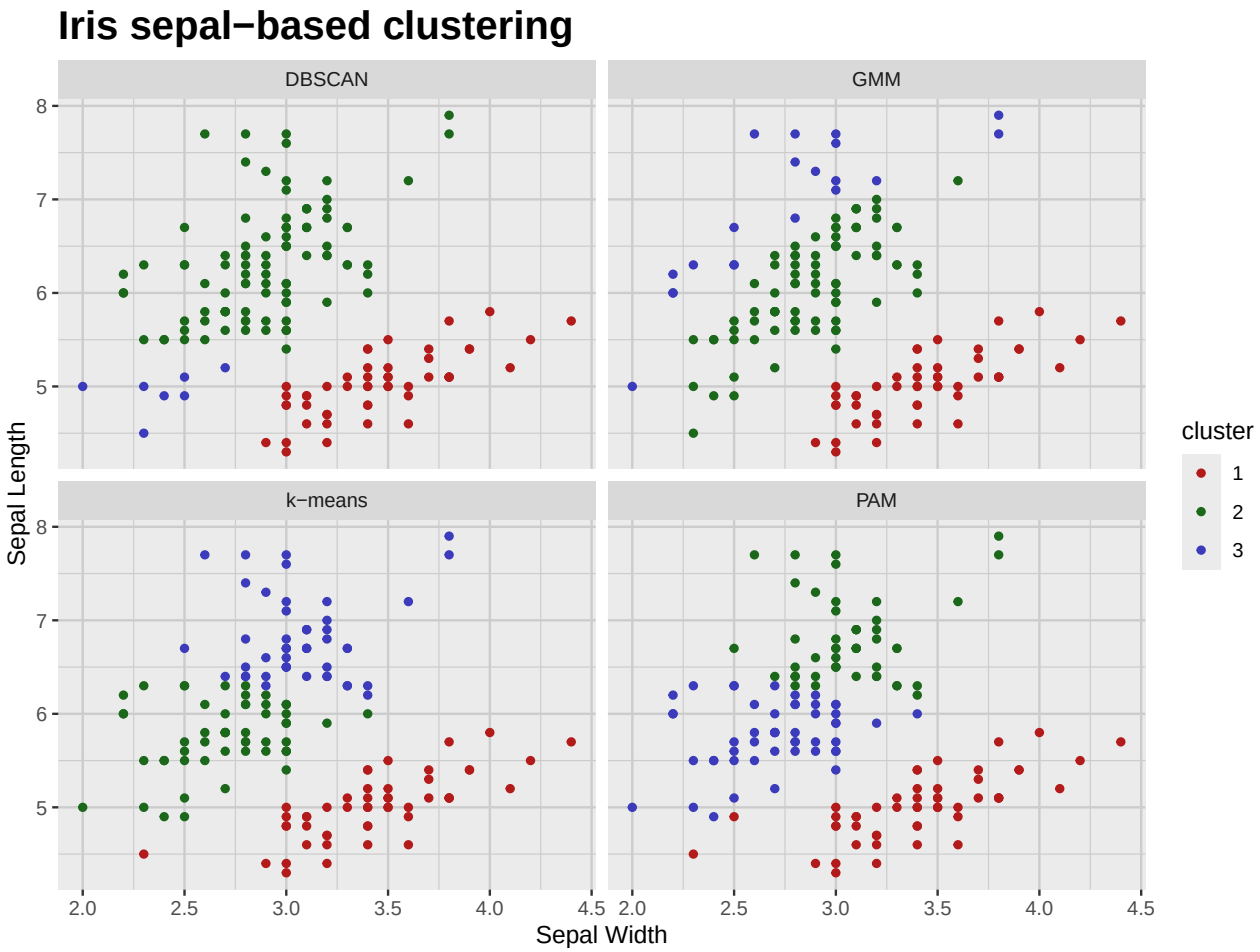

Figure 10.1: Clustering results on the iris dataset using Sepal.Width and Sepal.Length only. Notice the different shapes of the clusters obtained by different methods.

Part V

Time Series Analysis

Chapter 11

Stationary Time Series

Chapter 12

Non-Stationary Time Series